

İleri Seviye C

ve Linux Sistem Programlama Rehberi

İşaretçilerden Sistem Çağrılarına Kapsamlı Başvuru

Bu belge C dilinin ileri konularını (işaretçiler, birleşimler, fonksiyon işaretçileri, dinamik bellek, önişlemci) ve Linux sistem programlamayı (dosya G/Ç, süreçler, sinyaller, IPC, iş parçacıkları, soketler) **çok sayıda çalışan kod örneğiyle** adım adım anlatır.

Kapsam — Kısım 1 (İleri C): bellek modeli • saklama sınıfları • işaretçiler (temelden ileriye) • işaretçi aritmetiği • diziler & bellek düzeni • dinamik bellek (malloc/free) • yapılar & hizalama • birleşimler & tagged union • enum & bit alanları • fonksiyon işaretçileri & callback • değişken argümanlı fonksiyonlar • önişlemci & makrolar • jenerik programlama • bağlı liste / yığın / kuyruk.

Kapsam — Kısım 2 (Linux Sistem Programlama): sistem çağrıları & errno • düşük seviye dosya G/Ç • dosya sistemi & stat • süreçler (fork/exec/wait) • sinyaller • borular & FIFO • System V & POSIX IPC • paylaşımlı bellek • pthreads & senkronizasyon • TCP/UDP soketleri • mmap • zaman & zamanlayıcılar.

Hazırlanma Tarihi: 9 Temmuz 2026

GCC / glibc • POSIX • Linux x86-64

Contents

1 Giriş: C'nin Bellek Modeli ve Derleme Süreci	5
1.1 Derleme Aşamaları	5
1.2 Bir Programın Bellek Düzeni	5
2 Derleme Zamanı ve Çalışma Zamanı: Bellek Ne Zaman Oluşur?	6
2.1 Genel Bakış: Hangi Bilgi Ne Zaman Belirlenir?	7
2.2 Derleme Zamanında Ne Belirlenir?	7
2.3 Bağlama (Link) Zamanı: Adreslerin Atanması	8
2.4 Yükleme (Load) Zamanı: Program Belleğe Nasıl Gelir?	8
2.5 Çalışma Zamanında Stack: Yığın Çerçevesi	9
2.6 Çalışma Zamanında Heap: brk, mmap ve Talep Üzerine Sayfalama	9
2.7 Somut İnceleme: Canlı Bellek Haritası	10
3 Bellek, Saklama Sınıfları ve Nitelikler	10
3.1 Saklama Sınıfları (Storage Classes)	10
3.2 const, volatile ve restrict Nitelikleri	11
4 İşaretçiler: Temelden İleri Seviyeye	11
4.1 Temel Kavram: Adres ve Yönelme	11
4.2 NULL, Wild ve Dangling İşaretçiler	12
4.3 İşaretçi Aritmetiği	12
4.4 Diziler ve İşaretçiler: Aynı Değiller	13
4.5 İşaretçiden İşaretçiye (Double Pointer)	13
4.6 void İşaretçileri (Genel İşaretçi)	14
4.7 const ve İşaretçiler — Dört Kombinasyon	14
5 İleri İşaretçi Teknikleri	15
5.1 Karmaşık Bildirimleri Okuma: Sağ-Sol Kuralı	15
5.2 Diziyi Gösteren İşaretçi: <code>int (*p)[N]</code>	15
5.3 Çok Boyutlu Diziler ve İşaretçiye Çözülme	16
5.4 İşaretçi Takma Adı (Aliasing) ve Strict Aliasing Kuralı	16
5.5 Bellek Hizalama: <code>alignof</code> , <code>alignas</code> , <code>aligned_alloc</code>	17
5.6 <code>uintptr_t</code> : İşaretçi-Tamsayı Dönüşümü ve Etiketleme	17
5.7 <code>offsetof</code> ve <code>container_of</code> : Gömülü (Intrusive) Veri Yapıları	17
5.8 Opak İşaretçiler: Bilgi Gizleme ve Soyut Veri Tipleri	18
5.9 İleri Fonksiyon İşaretçileri: Bağlam Taşıyan Callback	19
5.10 <code>restrict</code> ile Optimizasyon ve Örtüşme (Overlap)	19
5.11 Çok Katmanlı <code>const</code> ve <code>const char **</code> Tuzağı	20
6 Dinamik Bellek Yönetimi	20
6.1 <code>malloc</code> , <code>calloc</code> , <code>realloc</code> , <code>free</code>	20
6.2 Yaygın Bellek Hataları	21
7 Yapılar (struct), Hizalama ve typedef	22
7.1 Tanım, Erişim ve typedef	22
7.2 Bellek Hizalama ve Dolgu (Padding)	22
7.3 Kendine Referanslı Yapılar	23
7.4 Esnek Dizi Üyesi (Flexible Array Member, C99)	23

8 Birlikler (union) ve Enum	24
8.1 union: Aynı Belleği Paylaşan Üyeler	24
8.2 Type Punning: Bir Float'ın Bitlerini Görme	24
8.3 Tagged Union (Etiketli Birleşim)	24
8.4 enum: İsimli Sabitler	25
8.5 Bit Alanları (Bit Fields)	25
8.6 Bit İşlemleri ve Bayrak Maskeleri	25
9 Fonksiyon İşaretçileri ve Callback'ler	26
9.1 Söz Dizimi	26
9.2 Callback: qsort ile Sıralama	27
9.3 Fonksiyon İşaretçisi Dizisi (Dispatch Table)	27
9.4 typedef ile Okunabilir Fonksiyon İşaretçisi	28
10 Değişken Argümanlı Fonksiyonlar (Variadic)	28
11 Önışlemci ve Makrolar	29
11.1 Nesne ve Fonksiyon Benzeri Makrolar	29
11.2 Koşullu Derleme ve Include Guard	29
11.3 Stringize (#) ve Token Paste (##)	29
11.4 Faydalı Hazır Makrolar	29
11.5 X-Macros: Tek Kaynaktan Çoklu Üretim	30
12 Jenerik Programlama Teknikleri	30
12.1 void * ile Jenerik Kap	31
12.2 _Generic ile Tipe Göre Seçim (C11)	31
13 Uygulama: İşaretçilerle Veri Yapıları	31
13.1 Tekli Bağlı Liste	31
13.2 Dinamik Yığın (Stack) — realloc ile Büyüyen Dizi	32
14 Sistem Çağrılar, Kernel ve errno	34
14.1 Kullanıcı Alanı ve Çekirdek Alanı	34
14.2 Hata Yönetimi: errno	34
15 Düşük Seviye Dosya G/Ç	35
15.1 open, read, write, close	35
15.2 lseek ve Dosya Konumu	36
15.3 dup, dup2 ve Betimleyici Yönlendirme	37
15.4 Betimleyici mi, Akış mı?	37
16 İleri Dosya G/Ç	37
16.1 Konumsal G/Ç: pread ve pwrite	37
16.2 Dağınık/Toplu G/Ç: readv ve writev (Scatter-Gather)	38
16.3 Dosya Bayraklarını Yönetme: fcntl	38
16.4 Dosya Kilitleme (fcntl F_SETLK)	39
16.5 Kalıcılık: fsync, fdatasync	39
16.6 Güvenli Geçici Dosya: mkstemp	40
16.7 Verimli Kopyalama: sendfile	40
16.8 stdio Tamponlaması: setvbuf	40

17 Dosya Sistemi: stat, Dizinler, İzinler	41
17.1 stat ile Dosya Bilgisi	41
17.2 Dizin Okuma (opendir/readdir)	42
17.3 İzinler: chmod, access, umask	42
17.4 Dizin ve Bağlantı İşlemleri	42
17.5 Dizin Ağacında Gezinme (nftw)	43
18 G/Ç Çoğullama: select, poll, epoll	44
18.1 Neden Gerekli?	44
18.2 select	44
18.3 poll	44
18.4 epoll — Ölçeklenebilir Linux Çözümü	45
19 Süreçler: fork, exec, wait	46
19.1 fork: Süreç Çoğaltma	46
19.2 exec: Program Değiştirme	47
19.3 wait, waitpid ve Çıkış Durumu	47
20 İleri Süreç Yönetimi	48
20.1 exec Ailesinin Tümü	48
20.2 Ortam Değişkenleri	48
20.3 Çıkış Kancaları: atexit ve _exit	49
20.4 Süreç Grupları ve Oturumlar	49
20.5 Arka Plan Servisi (Daemon) Oluşturma	50
20.6 Kaynak Sınırları: getrlimit / setrlimit	50
21 Terminal Komutları ve Program Çalıştırma	51
21.1 system: En Basit Yol (ve Tehlikeleri)	51
21.2 popen: Komut Çıktısını Okuma	51
21.3 Elle Çıktı Yakalama: fork + exec + pipe	51
21.4 Basit Bir Kabuk (Mini-Shell)	52
22 Sinyaller	53
22.1 Yaygın Sinyaller	53
22.2 sigaction ile Sinyal Yakalama	53
22.3 kill ve alarm	54
22.4 SIGCHLD ile Zombi Önleme	54
23 Borular (Pipes) ve FIFO	55
23.1 Anonim Boru (pipe)	55
23.2 Boru + exec: Kabuğun " " Operatörü	55
23.3 Adlandırılmış Boru (FIFO)	56
23.4 Çift Yönlü İletişim (İki Boru)	56
23.5 N Komutluk Boru Hattı	57
23.6 SIGPIPE: Kapalı Boruya Yazma	58
23.7 popen ile Boru — Hızlı Alternatif	58
24 IPC: Paylaşımlı Bellek, Mesaj Kuyruğu, Semafor	58
24.1 POSIX Paylaşımlı Bellek (shm_open + mmap)	58
24.2 POSIX Semafor	59
24.3 POSIX Mesaj Kuyruğu	60

25 İş Parçacıkları (POSIX Threads)	60
25.1 İş Parçacığı Oluşturma	60
25.2 Yarış Durumu (Race Condition)	61
25.3 Mutex ile Koruma	61
25.4 Koşul Değişkeni (Condition Variable)	62
25.5 İş Parçacığı mı, Süreç mi?	63
26 Atomik İşlemler (Atomics)	63
26.1 <code>_Atomic</code> ve <code><stdatomic.h></code>	63
26.2 Açık Atomik Fonksiyonlar	64
26.3 Compare-and-Swap (CAS): Kilitli Programlamanın Temeli	64
26.4 Bellek Sıralaması (Memory Ordering)	65
26.5 <code>atomic_flag</code> ile Spinlock	66
26.6 GCC/Clang Yerleşikleri (<code>__atomic</code>)	67
26.7 <code>volatile</code> Atomik Değildir!	67
26.8 Ne Zaman Hangisi?	68
27 Ağ Programlama: Soketler	68
27.1 TCP Sunucu	68
27.2 TCP İstemci	69
27.3 UDP: Bağlantısız İletişim	70
28 mmap: Bellek Eşleme	70
29 Zaman ve Zamanlayıcılar	71
30 GDB ile Hata Ayıklama	71
30.1 Hazırlık: Doğru Derleme	72
30.2 Programı Çalıştırma ve Durdurma	72
30.3 Kesme Noktaları (Breakpoints)	72
30.4 Adım Adım Yürütme (Stepping)	73
30.5 Veriyi İnceleme (Print, Examine)	73
30.6 Çağrı Yığını (Backtrace)	73
30.7 TUI: Metin Kullanıcı Arayüzü ve Layout'lar	74
30.8 Core Dump ve Çalışan Sürece Bağlanma	75
30.9 Diğer Faydalı Komutlar	75
30.10.gdbinit ile Otomasyon	75
31 Bağlayıcı (Linker) Derinlemesine: ld, Kütüphaneler ve Linker Script	76
31.1 gcc ile ld ilişkisi: Kim Kimi Çağırır?	76
31.2 Statik Kütüphane Oluşturma (.a)	76
31.3 Paylaşımlı Kütüphane Oluşturma (.so)	76
31.4 Sık Kullanılan Bağlama Bayrakları	77
31.5 Linker Script Nedir, Ne Zaman Gerekir?	77
31.6 Minimal Linker Script Anatomisi	78
31.7 Sembol Görünürlüğünü Denetleme (version script)	79
32 Ek: Derleme, Hata Ayıklama ve Araçlar	79

KISIM 1

İleri Seviye C Programlama

1. Giriş: C'nin Bellek Modeli ve Derleme Süreci

C, donanıma yakın çalışan, belleği doğrudan yöneten bir dildir. İleri C'yi anlamamanın anahtarı, programın çalışırken belleği nasıl kullandığını zihinde canlandırabilmektir. Bu bölüm, sonraki tüm konuların üzerine kurulacağı temeli atar.

1.1 Derleme Aşamaları

Bir .c dosyası çalıştırılabilir hale gelene kadar dört aşamadan geçer:

Aşama	Araç	Ne yapar?
Önişleme	cpp	#include, #define, #ifdef işlenir; saf C üretilir.
Derleme	cc1	C kodu mimariye özgü assembly'ye çevrilir (.s).
Assembly	as	Assembly, nesne koduna (.o) çevrilir.
Bağlama	ld	Nesne dosyaları + kütüphaneler birleştirilip çalıştırılabilir üretilir.

```

1 $ gcc -E main.c -o main.i      # yalnızca önişleme
2 $ gcc -S main.c -o main.s      # assembly üret
3 $ gcc -c main.c -o main.o      # nesne dosyası üret
4 $ gcc main.c -o main           # tüm aşamalar + bağlama
5
6 # Rehber boyunca önerilen derleme bayrakları:
7 $ gcc -std=c11 -Wall -Wextra -g -O2 main.c -o main
8 # -std=c11 : C11 standardı      -Wall -Wextra : tüm uyarılar
9 # -g       : hata ayıklama bilgisi -O2 : optimizasyon

```

1.2 Bir Programın Bellek Düzeni

Çalışan bir C programı, sanal bellekte birbirinden ayrı bölgelere yerleşir. Bu düzeni bilmek işaretçileri, static değişkenleri ve segfault'ları anlamamanın temelidir.

Yüksek Adresler	
Stack (Yığın)	Yerel değişkenler, fonksiyon parametreleri, dönüş adresleri. Aşağı doğru büyür.
↓ boş alan ↑	
Heap (Öbek)	malloc/calloc ile ayrılan dinamik bellek. Yukarı doğru büyür.
BSS	İklendirilmemiş global/static değişkenler (sıfırlanır).
Data	İklendirilmiş global/static değişkenler.
Text (Kod)	Makine kodu; salt-okunur.
Düşük Adresler	

Değişkenlerin nerede oturduğunu görme

Aşağıdaki program her değişken türünün adresini yazdırır. Adresleri karşılaştırdığında segmentlerin ayrıldığını görürsün.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int global_init = 5;      // Data segmenti
5 int global_zero;         // BSS segmenti
6
7 int main(void) {
8     int local = 1;        // Stack
9     static int static_var = 2; // Data
10    int *dynamic = malloc(sizeof(int)); // Heap
11
12    printf("Text (main) : %p\n", (void*)main);
13    printf("Data (global) : %p\n", (void*)&global_init);
14    printf("BSS (sifir) : %p\n", (void*)&global_zero);
15    printf("Heap (malloc) : %p\n", (void*)dynamic);
16    printf("Stack (yerel) : %p\n", (void*)&local);
17    printf("static : %p\n", (void*)&static_var);
18
19    free(dynamic);
20    return 0;
21 }
```

Bilgi

Stack otomatik yönetilir (fonksiyon bitince temizlenir), heap ise *senin sorumluluğundadır*: malloc ile aldığını free ile geri vermelisin. Bu ayrım, ileri C'nin en kritik konusudur.

2. Derleme Zamanı ve Çalışma Zamanı: Bellek Ne Zaman Oluşur?

Bir programın belleği tek bir anda ortaya çıkmaz. Bazı kararlar *derleme zamanında* (compile-time) betona dökülür, bazıları *bağlama zamanında* (link-time) çözülür, bir kısmı program *yüklenirken* (load-time) hazırlanır ve geri kalanı ancak *çalışma zamanında* (run-time) oluşur. Bu eksen ayırt edebilmek, “neden bu değişkenin adresi her çalıştırmada değişiyor?” veya “bu malloc aslında ne yapıyor?” gibi soruların cevabıdır.

2.1 Genel Bakış: Hangi Bilgi Ne Zaman Belirlenir?

Zaman	Ne belirlenir / ne oluşur?
Derleme	Tip boyutları (<code>sizeof</code>), sabit ifadeler, yerel değişkenlerin çerçevedeki <i>offsetleri</i> , global/static nesnelerin hangi section'a gireceği, sembol tablosu, kod (<code>.text</code>).
Bağlama	Semboller çözülür (relocation), section'lar birleştirilir, her sembole <i>sanal adres</i> atanır (statik bağlamada nihai, dinamikte görelî).
Yükleme	ELF segmentleri mmap ile eşlenir, <code>.bss</code> sıfırlanır, ASLR taban adresleri seçer, dinamik bağlayıcı kütüphaneleri bağlar.
Çalışma	Stack çerçeveleri push/pop edilir, heap <code>brk/mmap</code> ile büyür, sayfalar talep üzerine fiziksel belleğe düşer.

Anahtar sezgi

Derleyici *düzeni* (layout: boyutlar ve offsetler) belirler; *gerçek adresler* ve *fiziksel bellek* çalışma zamanına kadar var olmaz. Bu yüzden `sizeof` derleme zamanı sabitidir ama `&x` çalışma zamanı değeridir.

2.2 Derleme Zamanında Ne Belirlenir?

Derleyici tek bir *çeviri birimini* (translation unit: önışlemenden geçmiş `.c`) işler. Bu aşamada olanlar:

- **Sabit katlama (constant folding):** Derleme zamanı sabit ifadeler derlenirken hesaplanır; çalışma zamanında çarpma/toplama yapılmaz.
- **Tip ve boyut çözümü:** `sizeof`, `_Alignof`, dizi boyutları tamsayı sabitine dönüşür.
- **Yerleşim planı:** Global/static nesneler değerine göre `.data` (ilklenmiş), `.bss` (sıfır) veya `.rodata` (salt-okunur) section'ına atanır; yerel değişkenlere çerçeve içinde *offset* verilir (henüz adres değil).
- **Sembol tablosu:** Tanımlı ve dışa bağımlı (undefined) semboller nesne dosyasına yazılır.

```

1 int saniye = 24 * 60 * 60; // .data: derleyici 86400 yazar (folding)
2 int sayac; // .bss : yer ayrılır ama dosyada yer kaplamaz
3 const char *msg = "merhaba"; // isaretci .data'da, "merhaba" .rodata'da
4 char buf[4096]; // .bss : 4096 bayt, ikili dosyayı büyütmez
5
6 int kare(int x) { return x * x; } // .text: makine kodu

```

Section'ları nesne dosyasında görme

Derle ama bağlama, sonra araçlarla içine bak.

```

1 $ gcc -c ornek.c -o ornek.o
2
3 $ size ornek.o # section boyutlari
4 text data bss dec hex filename
5 123 12 4100 4235 108b ornek.o
6
7 $ nm ornek.o # semboller (T=text, D=data, B=bss, U=cozulmemis)

```



```

8 0000000000000000 T kare
9 0000000000000000 D saniye
10 0000000000000000 B sayac
11             U printf
12
13 $ readelf -S ornek.o      # section basliklari (.text .data .bss .rodata ...)
14 $ objdump -d ornek.o     # makine kodunu disassemble et

```

Dikkat

.bss, ikili dosyada yalnızca bir “boyut” olarak durur; 4 KB’lık bir global dizi diskteki dosyayı büyütmez, ama çalışma zamanında bellekte yer kaplar. .data ise başlangıç değerlerini dosyada taşıdığı için dosyayı büyütür.

2.3 Bağlama (Link) Zamanı: Adreslerin Atanması

Bağlayıcı (ld) tüm .o dosyalarını ve kütüphaneleri birleştirir:

- **Sembol çözümü:** Bir dosyadaki `U printf` gibi çözülmemiş referanslar, tanımın bulunduğu yere bağlanır. Bulunamazsa *undefined reference* hatası burada çıkar.
- **Relocation:** Aynı türden section’lar birleştirilir ve her sembole nihai sanal adres verilir; koddaki adres yer tutucuları düzeltilir.
- **Statik vs dinamik:** Statik bağlamada kütüphane kodu ikiliye kopyalanır (adresler kesinleşir). Dinamik bağlamada yalnızca bir referans bırakılır; asıl bağlama yükleme zamanına ertelenir.

```

1 $ ldd ./program          # hangi paylasimli kutuphanelere bagli?
2     linux-vdso.so.1
3     libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
4     /lib64/ld-linux-x86-64.so.2  # dinamik baglayici (yukleyici)
5
6 $ gcc main.c -static -o program # her seyi ikiliye kopyala (buyuk ama bagimsiz)

```

2.4 Yükleme (Load) Zamanı: Program Belleğe Nasıl Gelir?

execve sistem çağrısı çalıştığında çekirdek ve dinamik bağlayıcı sırayla şunları yapar:

1. Çekirdek ELF başlığını okur; .text, .data segmentlerini dosyadan mmap ile sanal belleğe eşler (henüz fiziksel RAM’e kopyalanmaz — bkz. talep üzerine sayfalama).
2. .bss için sıfırlanmış sayfalar ayrılır.
3. **ASLR** (Address Space Layout Randomization) etkinse stack, heap, kütüphaneler ve (PIE ikililerde) kod *rastgele* taban adreslerine yerleştirilir — bu yüzden adresler her çalıştırmada değişir.
4. Dinamik bağlayıcı (ld.so) paylaşımlı kütüphaneleri eşler ve sembolleri **GOT/PLT** üzerinden bağlar (çoğu zaman *lazy*: fonksiyon ilk çağrıldığında).
5. Kontrol `_start`’a geçer; o `__libc_start_main`’i, o da senin `main`’ini çağırır.

```

1 $ setarch -R ./program  # ASLR’yi kapatarak calistir (adresler sabitlenir)
2 $ ltrace ./program      # kutuphane cagrilarini izle (lazy binding gorulur)
3 $ LD_DEBUG=bindings ./program # dinamik baglayicinin ne yaptigini gor

```

Bilgi

“Adres neden her çalıştırmada farklı?” sorusunun cevabı ASLR’dir. Aynı programı iki kez çalıştırıp `&local` yazdırırsan farklı değerler görürsün; `setarch -R` ile aynılaşırlar.

2.5 Çalışma Zamanında Stack: Yığın Çerçevesi

Her fonksiyon çağrısında bir *çerçeve* (stack frame) oluşur: dönüş adresi, kaydedilen register’lar ve yerel değişkenler. Derleyici bunların *ofsetlerini* derleme zamanında bilir; asıl adresler çalışma zamanında, `rsp/rbp` register’larına göre belirlenir.

```
1 int toplama(int a, int b) {
2     int s = a + b;    // 's' çerçevede rbp'ye göre sabit ofsette
3     return s;
4 }
```

```
1 $ gcc -O0 -c toplama.c && objdump -d toplama.o
2 toplama:
3     push    %rbp                # eski çerçeve tabanını sakla (prologue)
4     mov     %rsp,%rbp           # yeni çerçeve tabanı
5     mov     %edi,-0x14(%rbp)    # a -> yerel ofset
6     mov     %esi,-0x18(%rbp)    # b -> yerel ofset
7     ...
8     pop     %rbp                # çerçeveyi geri al (epilogue)
9     ret
```

Dikkat

Stack sınırlıdır (tipik olarak 8 MB, `ulimit -s`). Çok derin özyineleme veya devasa yerel dizi (`int buf[1000000];`) *stack overflow* ve *segfault* üretir. Büyük tamponları heap’ten al.

2.6 Çalışma Zamanında Heap: `brk`, `mmap` ve Talep Üzerine Sayfalama

Heap tamamen çalışma zamanı olgusudur. `malloc` bir sistem çağrısı *değildir*; kullanıcı alanında bir ayırıcıdır (glibc’de `ptmalloc`) ve çekirdekten toptan bellek ister:

- Küçük istekler için `brk/sbrk` ile heap sınırını yukarı iter ve bu bloktan parçalar dağıtır.
- Büyük istekler için (varsayılan eşik ≈ 128 KB, `M_MMAP_THRESHOLD`) doğrudan `mmap` ile anonim bölge alır; `free` bunu `munmap` ile geri verir.
- **Talep üzerine sayfalama (demand paging):** `malloc` sana *sanal* adres verir; fiziksel RAM ancak o sayfaya ilk *dokunduğunda* bir *page fault* ile atanır. Bu yüzden büyük bir `malloc` anında RAM tüketmez.

```
1 $ strace -e trace=brk,mmap,munmap ./program    # malloc'un altındaki çağrılar
2 $ /usr/bin/time -v ./program | grep resident  # RSS: gerçekten kullanılan RAM
```

VSZ vs RSS

`top/ps` çıktısındaki **VSZ** ayrılan *sanal* bellektir (dokunulmamış `malloc` dahil); **RSS** ise fiilen RAM’e düşmüş sayfalardır. 1 GB `malloc` edip hiç yazmazsan VSZ büyür, RSS neredeyse sabit kalır.

2.7 Somut İnceleme: Canlı Bellek Haritası

Çalışan bir sürecin tüm bölgelerini /proc altından okuyabilirsin — teoriyi gözünle görmenin en iyi yolu:

```
1 $ cat /proc/self/maps          # veya /proc/<PID>/maps
2 555...000-555...000 r-xp  ...  /path/program  # .text (kod, çalıştırılabilir)
3 555...000-555...000 r--p  ...  /path/program  # .rodata (salt-okunur)
4 555...000-555...000 rw-p  ...  /path/program  # .data + .bss
5 555...000-556...000 rw-p  ...  [heap]          # brk ile büyüten heap
6 7f5...000-7f5...000 r-xp  ...  libc.so.6       # paylaşımlı kutuphane
7 7ff...000-7ff...000 rw-p  ...  [stack]         # stack (aşağı büyür)
8
9 $ pmap -x <PID>                # aynı bilginin özetlenmiş, okunaklı hali
```

İpucu

İzin sütunu (r-xp, rw-p) segmentin doğasını ele verir: kod r-x (yazılamaz, çalıştırılabilir), veri rw- (çalıştırılmaz). Bu ayrım güvenliğin temelidir: W^X ilkesi bir bölgenin aynı anda hem yazılabilir hem çalıştırılabilir olmasını engeller.

3. Bellek, Saklama Sınıfları ve Nitelikler

3.1 Saklama Sınıfları (Storage Classes)

Saklama sınıfı, bir değişkenin *ömrünü* (lifetime) ve *görünürlüğünü* (scope) belirler.

Anahtar	Ömür	Görünürlük	Açıklama
auto	blok	blok	Varsayılan; yerel değişkenler. Neredeyse hiç yazılmaz.
register	blok	blok	Register'da tutulması <i>önerilir</i> (modern derleyiciler yok sayar).
static	program	sınırlı	Değeri çağrılar arası korunur; dosya kapsamında dışarı kapalıdır.
extern	program	global	Başka dosyada tanımlı değişkene/fonksiyona bildirim.

static ile durum saklama

Fonksiyon içi static değişken, çağrılar arasında değerini korur.

```
1 #include <stdio.h>
2
3 int counter(void) {
4     static int n = 0;    // yalnızca ilk çağrıda ilklenir
5     return ++n;
6 }
7
8 int main(void) {
9     printf("%d %d %d\n", counter(), counter(), counter()); // 1 2 3 (bkz. not)
10    return 0;
11 }
```

Dikkat

Yukarıda `printf` argümanlarının değerlendirilme sırası *tanımsızdır*; farklı derleyiciler "3 2 1" de yazabilir. Yan etkili ifadeleri tek bir `printf`'e sıralı biçimde koymaktan kaçın.

3.2 const, volatile ve restrict Nitelikleri

- `const`: değer değiştirilemez (salt-okunur). Derleyici zorlar.
- `volatile`: değer her erişimde bellekten okunur; derleyici optimize edip önbelleğe almaz. Donanım register'ları ve sinyal bayrakları için şarttır.
- `restrict` (C99): "bu işaretçi, gösterdiği belleğe erişen tek işaretçidir" sözü verir; derleyici agresif optimizasyon yapar.

```

1  const double PI = 3.14159;           // değiştirilemez
2  volatile sig_atomic_t flag = 0;      // sinyal handler'ı değiştirebilir
3
4  // restrict: src ve dst çakışmaz sözü -> daha hızlı kod
5  void copy(int *restrict dst, const int *restrict src, size_t n) {
6      for (size_t i = 0; i < n; i++) dst[i] = src[i];
7  }
```

const'un yeri anlamı değiştirir

`const int *p` → p'nin gösterdiği *değer* sabittir. `int *const p` → p'nin kendisi (adres) sabittir. `const int *const p` → ikisi de sabittir.

4. İşaretçiler: Temelden İleri Seviyeye

İşaretçi, bir *bellek adresini* tutan değişkendir. C'nin gücünün ve tehlikesinin kaynağıdır. Bu bölümü sindirmek, ileri C'nin yarısını halletmek demektir.

4.1 Temel Kavram: Adres ve Yönelme

- `&x` — x değişkeninin *adresini* verir (address-of).
- `*p` — p'nin gösterdiği adresteki *değeri* verir (dereference).
- `int *p` — "p, bir `int`'i gösteren işaretçidir".

```

1  #include <stdio.h>
2  int main(void) {
3      int x = 42;
4      int *p = &x;           // p, x'in adresini tutar
5
6      printf("x = %d\n", x);   // 42
7      printf("&x = %p\n", (void*)&x);
8      printf("p = %p\n", (void*)p); // &x ile aynı
9      printf("*p = %d\n", *p);  // 42 (p'nin gösterdiği değer)
10
11     *p = 100;                // x'i işaretçi üzerinden değiştir
```

```

12     printf("x = %d\n", x);        // 100
13     return 0;
14 }

```

Bilgi

İşaretçinin *tipi* önemlidir: `int *` bir `int` gösterir, `double *` bir `double`. Tip, `*p` ile kaç bayt okunacağını ve işaretçi aritmetiğinin adım boyunu belirler.

4.2 NULL, Wild ve Dangling İşaretçiler

- **NULL işaretçi:** hiçbir yeri göstermez (`p = NULL`). Yönelmek (`*p`) segfault üretir. Her zaman kontrol et.
- **Wild (başboş) işaretçi:** ilklenmemiş; rastgele bir adres tutar. Çok tehlikelidir. Bildirir bildirmez ilklendir.
- **Dangling (asılı) işaretçi:** serbest bırakılmış (`free`) ya da kapsamı bitmiş belleği gösterir.

```

1  int *wild;           // TEHLIKE: rastgele adres
2  int *safe = NULL;    // guvenli baslangic
3
4  int *dangling = malloc(sizeof(int));
5  free(dangling);      // bellek geri verildi
6  // *dangling = 5;    // HATA: dangling pointer -> tanimsiz davranis
7  dangling = NULL;     // free'den sonra NULL'la -> guvenli

```

Dikkat

Serbest bıraktıktan sonra işaretçiyi `NULL`'a ata. Böylece tekrar `free` (double-free) veya erişim (use-after-free) hataları segfault'a dönüşür ve fark edilir; sessiz bellek bozulması yaşanmaz.

4.3 İşaretçi Aritmetiği

İşaretçiye tamsayı eklemek, adresi *tip boyutu kadar* kaydırır. Yani `p + 1`, bir sonraki `int`'i gösterir — 4 bayt ileriye (`int` için), 1 bayt değil.

```

1  #include <stdio.h>
2  int main(void) {
3      int a[5] = {10, 20, 30, 40, 50};
4      int *p = a;           // p, a[0]'i gosterir (dizi -> isaretcie cozulur)
5
6      printf("%d\n", *p);    // 10
7      printf("%d\n", *(p + 1)); // 20 (p+1 -> a[1], 4 bayt ileri)
8      printf("%d\n", *(p + 3)); // 40
9      p++;                  // artik a[1]'i gosterir
10     printf("%d\n", *p);    // 20
11
12     // iki isaretci farki: aradaki ELEMEN sayisi
13     int *q = &a[4];
14     printf("fark = %ld\n", (long)(q - p)); // 3
15     return 0;
16 }

```

Dizi indeksleme aslında işaretçi aritmetiğidir

`a[i]` ifadesi tam olarak `*(a + i)` demektir. Bu yüzden ilginç ama geçerli olan `i[a]` yazımı da `a[i]` ile aynıdır! Çünkü toplama değişmelidir.

4.4 Diziler ve İşaretçiler: Aynı Değiller

Diziler çoğu bağlamda ilk elemanı gösteren işaretçiye "çözülür" (decay), ama *özdeş değildir*.

Dizi <code>int a[5]</code>	İşaretçi <code>int *p</code>
<code>sizeof(a) = 20</code> (tüm dizi)	<code>sizeof(p) = 8</code> (yalnız adres)
<code>a</code> yeniden atanamaz	<code>p</code> yeniden atanabilir
<code>&a</code> tipi <code>int(*)[5]</code>	<code>&p</code> tipi <code>int**</code>

```

1 #include <stdio.h>
2 void func(int arr[]) {                // aslında int *arr
3     // BURADA sizeof(arr) == sizeof(int*) == 8, dizi boyutu DEĞİL!
4     printf("fonksiyon içinde: %zu\n", sizeof(arr)); // 8
5 }
6 int main(void) {
7     int a[5];
8     printf("main içinde: %zu\n", sizeof(a)); // 20
9     func(a);
10    return 0;
11 }

```

Dikkat

Bir dizi fonksiyona geçince boyut bilgisi kaybolur (işaretçiye çözülür). Bu yüzden dizi boyutunu *ayrı bir parametre* olarak geçmelisin: `void f(int *arr, size_t n)`.

4.5 İşaretçiden İşaretçiye (Double Pointer)

Bir işaretçinin adresini tutan işaretçidir: `int **pp`. Kullanım alanları: bir fonksiyonun çağırının işaretçisini değiştirmesi ve işaretçi dizileri (örn. `char *argv[]`).

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Bir fonksiyonun cagiranin isaretcisini degistirmesi icin
5 // isaretcinin ADRESI (int**) gerekir.
6 void allocate(int **pp, int value) {
7     *pp = malloc(sizeof(int)); // cagiranin isaretcisine yaz
8     **pp = value;              // o bellege deger yaz
9 }
10
11 int main(void) {
12     int *p = NULL;
13     allocate(&p, 99);          // p'nin ADRESINI gecir
14     printf("%d\n", *p);        // 99
15     free(p);
16     return 0;
17 }

```

İşaretçi dizisi: string tablosu

char * dizisi, farklı uzunlukta stringleri verimli tutar.

```

1 #include <stdio.h>
2 int main(void) {
3     const char *days[] = {"Pzt", "Sal", "Car", "Per", "Cum"};
4     size_t n = sizeof(days) / sizeof(days[0]);
5     for (size_t i = 0; i < n; i++)
6         printf("%zu -> %s\n", i, days[i]);
7     return 0;
8 }

```

4.6 void İşaretçileri (Genel İşaretçi)

void * herhangi bir tipin adresini tutabilir; ama yönelmeden önce somut bir tipe *cast* edilmelidir. malloc bu yüzden void * döndürür.

```

1 #include <stdio.h>
2 void print_value(void *data, char type) {
3     switch (type) {
4         case 'i': printf("%d\n", *(int*)data); break;
5         case 'd': printf("%g\n", *(double*)data); break;
6         case 'c': printf("%c\n", *(char*)data); break;
7     }
8 }
9 int main(void) {
10     int i = 5; double d = 3.14; char c = 'A';
11     print_value(&i, 'i');
12     print_value(&d, 'd');
13     print_value(&c, 'c');
14     return 0;
15 }

```

Dikkat

void * üzerinde işaretçi aritmetiği standart C'de *yasaktır* (boyutu bilinmez). GCC bir eklenti olarak 1 bayt kabul eder, ama taşınabilir kod için önce char *'a cast et.

4.7 const ve İşaretçiler — Dört Kombinasyon

```

1 int x = 10, y = 20;
2 int *p1 = &x;           // her sey degisebilir
3 const int *p2 = &x;       // *p2 sabit (deger), p2 degisebilir
4 int *const p3 = &x;       // p3 sabit (adres), *p3 degisebilir
5 const int *const p4 = &x; // ikisi de sabit
6
7 // *p2 = 5;   // HATA: gosterilen deger sabit
8 p2 = &y;      // OK: isaretci baska yeri gosterebilir
9 *p3 = 5;      // OK: deger degistirilebilir
10 // p3 = &y;   // HATA: isaretci sabit

```

İpucu

Okuma kuralı: işaretçiyi *sağdan sola* oku. int *const p = "p is a const pointer to int". const int *p = "p is a pointer to const int".

5. İleri İşaretçi Teknikleri

Bu bölüm, temel işaretçilerin ötesine geçer: karmaşık bildirimler, diziyi gösteren işaretçiler, hizalama, takma ad (aliasing) kuralları, kernel tarzı gömülü veri yapıları, opak işaretçiler ve bağlam taşıyan callback'ler. Bunlar gerçek sistem kodunun (Linux çekirdeği, glibc, gömülü sistemler) günlük araçlarıdır.

5.1 Karmaşık Bildirimleri Okuma: Sağ-Sol Kuralı

C bildirimlerini çözmek için *isimden başla, önce sağa sonra sola* git; parantezler önceliği değiştirir. () ve [] operatörleri *'dan önce gelir.

Bildirim	Okunuşu
<code>int *p[10]</code>	10 elemanlı, her biri <code>int*</code> olan <i>dizi</i> .
<code>int (*p)[10]</code>	10 elemanlı bir <code>int</code> dizisini gösteren <i>işaretçi</i> .
<code>int *f(void)</code>	<code>int*</code> döndüren fonksiyon.
<code>int (*f)(void)</code>	<code>int</code> döndüren fonksiyonu gösteren işaretçi.
<code>int (*fp[4])(int)</code>	Her biri fonksiyon işaretçisi olan 4'lük dizi (jump table).
<code>int (*(fp)[5])(void)</code>	5'lik fonksiyon-işaretçisi dizisini gösteren işaretçi.
<code>char **argv</code>	<code>char*</code> dizisinin ilk elemanını gösteren işaretçi.

İpucu

cdecl aracı bu bildirimleri Türkçe/İngilizce açıklar: `echo 'explain int (*p)[10]' | cdecl`. typedef kullanmak, karmaşık tipleri okunur parçalara böler ve en iyi çözümdür.

5.2 Diziyi Gösteren İşaretçi: `int (*p)[N]`

Diziyi gösteren işaretçi, iki boyutlu dizilerde satır satır ilerlemenin doğru yoludur. `p + 1`, *tüm bir satır* kadar (N eleman) ilerler.

```

1 #include <stdio.h>
2 int main(void) {
3     int matrix[3][4] = {
4         { 1,  2,  3,  4},
5         { 5,  6,  7,  8},
6         { 9, 10, 11, 12},
7     };
8
9     // 'row', 4 int'lik bir diziyi gösterir. p++ -> sonraki SATIR.
10    int (*row)[4] = matrix;           // matrix -> &matrix[0]
11    for (int i = 0; i < 3; i++) {
12        for (int j = 0; j < 4; j++)
13            printf("%3d ", (*row)[j]);    // (*row) = matrix[i]
14        printf("\n");
15        row++;                          // 4 int (16 bayt) ileri: sonraki satır
16    }
17    return 0;
18 }
```


Dikkat

`int (*p)[4]` ile `int *p[4]` tamamen farklıdır. İlki bir işaretçidir (8 bayt), ikincisi 4 işaretçilik bir dizidir (32 bayt). Parantez şart.

5.3 Çok Boyutlu Diziler ve İşaretçiye Çözülme

`int a[3][4]` bellekte *bitişik* 12 `int`'tir (sıra-öncelikli). Bir fonksiyona geçince ilk boyut düşer ve `int (*)[4]`'e çözülür — ikinci boyut korunmalıdır.

```

1 #include <stdio.h>
2 // 2B diziyi doğru gecirme: satir sayisi ayri, sutun tipte gomulu
3 void print_matrix(int rows, int cols, int m[][cols]) { // VLA parametresi (C99)
4     for (int i = 0; i < rows; i++) {
5         for (int j = 0; j < cols; j++)
6             printf("%d ", m[i][j]); // m[i][j] = (*(m+i)+j)
7         printf("\n");
8     }
9 }
10 int main(void) {
11     int a[2][3] = {{1,2,3},{4,5,6}};
12     print_matrix(2, 3, a); // int(*)[3] olarak gecer
13     return 0;
14 }
```

İndekslemenin açılımı

`a[i][j]` ifadesi `*(a + i) + j` demektir: önce `i`. satırın adresi, sonra o satırda `j`. eleman. Derleyici sıra-öncelikli düzende `i * sutun + j` ofsetini hesaplar.

5.4 İşaretçi Takma Adı (Aliasing) ve Strict Aliasing Kuralı

Derleyici, *farklı tipteki* iki işaretçinin aynı belleği göstermediğini varsayarak optimizasyon yapar (strict aliasing). Bunu ihlal etmek sessiz hatalara yol açar.

```

1 #include <stdint.h>
2 #include <string.h>
3
4 float example(void) {
5     float f = 1.0f;
6
7     // YANLIS: strict aliasing ihlali -> tanimsiz davranis
8     // uint32_t u = *(uint32_t *)&f;
9
10    // DOGRU: memcpy ile tip donusumu (derleyici bunu optimize eder)
11    uint32_t u;
12    memcpy(&u, &f, sizeof u); // bitleri guvenle kopyala
13    u |= 0x80000000; // isaret bitini set et (negatifle)
14    memcpy(&f, &u, sizeof f);
15    return f; // -1.0f
16 }
```

Bilgi

`char *`, `unsigned char *` ve `void *` her tipe takma ad olabilir — kural onlar için geçerli değildir. Bu yüzden `memcpy` (bayt bazlı) her zaman güvenlidir. Tip cezalandırmanın (type punning) diğer geçerli yolu, daha önce gördüğümüz `union`'dır.

5.5 Bellek Hizalama: alignof, alignas, aligned_alloc

Her tipin bir hizalama gereksinimi vardır (bir int genelde 4'ün katı adreste durur). C11 bunu açıkça kontrol etmeyi sağlar — SIMD, DMA ve donanım tamponları için gereklidir.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdalign.h>
4
5 int main(void) {
6     printf("int hizasi    : %zu\n", alignof(int));    // genelde 4
7     printf("double hizasi: %zu\n", alignof(double)); // genelde 8
8
9     // 64 baytlık hizalı değişken (ornegin cache satiri)
10    alignas(64) char buffer[128];
11    printf("tampon adresi %% 64 = %ld\n", (long)((uintptr_t)buffer % 64)); // 0
12
13    // Hizalı dinamik ayırma (boyut, hizalamanın kati olmalı)
14    void *p = aligned_alloc(64, 256); // 64-hizalı, 256 bayt
15    if (p) {
16        printf("aligned_alloc %% 64 = %ld\n", (long)((uintptr_t)p % 64)); // 0
17        free(p);
18    }
19    return 0;
20 }
21 // Derleme: gcc -std=c11 bu.c

```

5.6 uintptr_t: İşaretçi-Tamsayı Dönüşümü ve Etiketleme

<stdint.h>'deki uintptr_t, bir işaretçiyi güvenli tutabilen tamsayı tipidir. Hizalanmış işaretçilerin kullanılmayan alt bitlerine bayrak gömmek (pointer tagging) ileri bir tekniktir.

```

1 #include <stdio.h>
2 #include <stdint.h>
3 int main(void) {
4     int data = 42;
5     int *p = &data;
6
7     // 4-hizalı isaretcinin alt 2 biti daima 0'dir -> etiket için kullan
8     uintptr_t tagged = (uintptr_t)p | 1u;    // alt bite bayrak koy
9
10    int flag = tagged & 1u;    // bayragi oku
11    int *clean = (int *) (tagged & ~(uintptr_t)3u); // isaretciyi geri al
12
13    printf("bayrak=%d, deger=%d\n", flag, *clean); // bayrak=1, deger=42
14    return 0;
15 }

```

Dikkat

Etiketleme yalnızca hizalamanın garanti ettiği bitlerde yapılabilir ve işaretçiyi kullanmadan önce bitler mutlaka temizlenmelidir. Yanlış kullanım segfault üretir; bu teknik yalnızca performans-kritik kodda tercih edilir.

5.7 offsetof ve container_of: Gömülü (Intrusive) Veri Yapıları

Linux çekirdeğinin klasik hilesi: bir yapı üyesinin adresinden, onu içeren dış yapının adresini bulmak. Bu, tek bir liste kodunu tüm tiplerde kullanmayı sağlar.

```

1 #include <stdio.h>
2 #include <stddef.h>           // offsetof
3
4 // Bir uye isaretcisinden, kapsayan yapıya ulas:
5 #define container_of(ptr, type, member) \
6     ((type *)((char *)(ptr) - offsetof(type, member)))
7
8 struct list_node { struct list_node *next; }; // genel liste dugumu
9
10 struct task {                // gercek veri: liste dugumunu ICINE gomer
11     int id;
12     struct list_node node;
13 };
14
15 int main(void) {
16     struct task g = { .id = 7 };
17     struct list_node *n = &g.node;           // yalnızca dugumu tutuyoruz
18
19     // Dugumden geri kapsayan goreve ulas:
20     struct task *owner = container_of(n, struct task, node);
21     printf("id = %d\n", owner->id);           // 7
22     printf("offset(dugum) = %zu\n", offsetof(struct task, node));
23     return 0;
24 }

```

Bilgi

`offsetof(tip, uye)`, bir üyenin yapının başından kaç bayt sonra başladığını verir. `container_of` bu ofseti işaretçiden çıkararak dış yapıya ulaşır. Bu desen "intrusive" liste/ağaç yapılarının ve `list_head` API'sinin temelidir.

5.8 Opak İşaretçiler: Bilgi Gizleme ve Soyut Veri Tipleri

Bir yapının *tanımını* başlıkta gizleyip yalnızca işaretçisini (handle) sunmak, C'de kapsülleme (encapsulation) sağlar. Kullanıcı içeriği göremez, yalnızca API fonksiyonlarını çağırır.

```

1 // ===== stack.h (kullaniciya acik) =====
2 typedef struct Stack Stack;           // OPAK: tanim gizli, sadece isim
3 Stack *stack_create(void);
4 void stack_push(Stack *, int);
5 int stack_pop(Stack *, int *);
6 void stack_destroy(Stack *);
7
8 // ===== stack.c (uygulama, gizli) =====
9 #include "stack.h"
10 #include <stdlib.h>
11 struct Stack {                        // tam tanim yalnızca BURADA gorunur
12     int *data;
13     size_t count, capacity;
14 };
15 Stack *stack_create(void) {
16     Stack *s = calloc(1, sizeof *s); // kullanıcı sizeof(Stack) bilemez
17     return s;
18 }
19 // ... stack_push / stack_pop / stack_destroy ...

```

İpucu

Opak işaretçi, ikili uyumluluğu (ABI) korur: yapının iç düzenini değiştiren bile kullanıcı kodunu yeniden derlemek gerekmez. FILE, DIR, pthread_t gibi standart tipler bu yüzden opakdır.

5.9 İleri Fonksiyon İşaretçileri: Bağlam Taşıyan Callback

Gerçek callback'ler genellikle bir void *userdata parametresiyle "bağlam" (context) taşır — C'de kapanış (closure) benzetiminin yoludur.

```

1 #include <stdio.h>
2
3 // Callback: her eleman + kullanıcı verisi alır
4 typedef void (*Action)(int element, void *userdata);
5
6 void for_each(const int *arr, size_t n, Action fn, void *userdata) {
7     for (size_t i = 0; i < n; i++)
8         fn(arr[i], userdata);
9 }
10
11 // userdata'yi bir toplam biriktirici olarak kullan
12 void accumulate(int e, void *userdata) {
13     *(long *)userdata += e;
14 }
15
16 int main(void) {
17     int a[] = {1, 2, 3, 4, 5};
18     long sum = 0;
19     for_each(a, 5, accumulate, &sum);    // bağlam: &sum
20     printf("toplam = %ld\n", sum);        // 15
21     return 0;
22 }
```

Fonksiyon İşaretçisi Döndüren Fonksiyon

```

1 #include <stdio.h>
2 typedef int (*BinOp)(int, int);
3 int add(int a, int b)    { return a + b; }
4 int multiply(int a, int b) { return a * b; }
5
6 // typedef ile temiz: bir işlem döndürür
7 BinOp choose(char op) { return (op == '+') ? add : multiply; }
8
9 // typedef'siz ham sözdizimi (aynı şey):
10 // int (*choose(char op))(int, int) { ... }
11
12 int main(void) {
13     BinOp f = choose('*');
14     printf("%d\n", f(6, 7));    // 42
15     return 0;
16 }
```

5.10 restrict ile Optimizasyon ve Örtüşme (Overlap)

restrict, işaretçilerin çakışmadığı sözünü verince derleyici gereksiz yeniden yüklemeleri (reload) atlar. Söz yalanlanırsa (bloklar örtüşürse) davranış tanımsızdır — memcpy bunu varsayar, memmove varsaymaz.

```

1 #include <stddef.h>
2 // restrict YOK: derleyici a[i] her dongude b'nin degistirmis olabilecegini
3 // varsayar ve tekrar okur -> yavas.
4 void scale_slow(int *a, const int *b, size_t n, int k) {
5     for (size_t i = 0; i < n; i++) a[i] = b[i] * k;
6 }
7 // restrict VAR: a ve b cakismaz sozu -> vektorlestirme mumkun.
8 void scale_fast(int *restrict a, const int *restrict b, size_t n, int k) {
9     for (size_t i = 0; i < n; i++) a[i] = b[i] * k;
10 }

```

5.11 Çok Katmanlı const ve const char ** Tuzakı

```

1 // Sezgiye aykiri: char** -> const char** donusumu GECERSIZDIR.
2 // Cunku izin verilseydi const'lugu delmek mumkun olurdu.
3 void f(const char **p);
4 char *arr[1];
5 // f(arr);          // UYARI/HATA: char** -> const char** guvenli degil
6
7 // Dogru imza: hem eleman hem isaretci const olmalı
8 void g(const char *const *p); // bu char** kabul eder

```

İpucu

Çok katmanlı işaretçilerde const, yalnızca *en dış* seviyede otomatik güvenli eklenir. İç seviyelerde de const isteniyorsa imzayı buna göre yaz: const char *const *.

6. Dinamik Bellek Yönetimi

Derleme anında boyutu bilinmeyen veriler için heap'ten çalışma zamanında bellek ayrılır. Bu, C'nin en güçlü ama en hataya açık alanıdır.

6.1 malloc, calloc, realloc, free

Fonksiyon

Görev

malloc(n)	n bayt ayırır; içerik <i>çöp</i> (ilklenmemiş). Başarısızsa NULL.
calloc(k,n)	k×n bayt ayırır ve <i>sıfırlar</i> . Taşma kontrolü yapar.
realloc(p,n)	p'yi n bayta büyütür/küçültür; içeriği korur, taşıyabilir.
free(p)	p ile ayrılan belleği geri verir. free(NULL) güvenlidir.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void) {
5     size_t n = 5;
6     int *arr = malloc(n * sizeof(int)); // ayir
7     if (arr == NULL) {                  // HER ZAMAN kontrol et
8         perror("malloc");

```

```

9     return 1;
10 }
11 for (size_t i = 0; i < n; i++) arr[i] = (int)(i * i);
12
13 // Buyut: 5 -> 10 eleman
14 int *grown = realloc(arr, 10 * sizeof(int));
15 if (grown == NULL) { // realloc basarisizsa
16     free(arr); // eski blok hala gecerli
17     return 1;
18 }
19 arr = grown; // dogru kullanim
20 for (size_t i = 5; i < 10; i++) arr[i] = (int)(i * i);
21
22 for (size_t i = 0; i < 10; i++) printf("%d ", arr[i]);
23 printf("\n");
24
25 free(arr); // geri ver
26 arr = NULL; // dangling'i onle
27 return 0;
28 }

```

Dikkat

`p = realloc(p, n)` yaygın ama tehlikeli bir kalıptır: `realloc` `NULL` dönerse eski işaretçiyi kaybedip *bellek sızıntısı* yaratırsın. Yukarıdaki gibi geçici bir değişken kullan.

6.2 Yaygın Bellek Hataları

Hata

Açıklama

Memory leak	<code>free</code> unutulur; program bellek tüketmeye devam eder.
Double free	Aynı blok iki kez <code>free</code> edilir; heap bozulur.
Use-after-free	Serbest bırakılmış belleğe erişim.
Buffer overflow	Ayrılan sınırın dışına yazma.
Uninitialized read	<code>malloc</code> 'tan gelen çöp değerın okunması.

İpucu

Bu hataları **Valgrind** ve **AddressSanitizer** yakalar:

```

valgrind -leak-check=full ./program
gcc -fsanitize=address -g program.c && ./a.out

```

2B dizi (matris) dinamik ayırma

Satır sayısı çalışma zamanında belliyse, işaretçi dizisi + her satır için ayrı blok kullanılır.

```

1 #include <stdlib.h>
2 int **matrix_create(size_t rows, size_t cols) {
3     int **m = malloc(rows * sizeof(int *)); // satir isaretcisi
4     if (!m) return NULL;
5     for (size_t i = 0; i < rows; i++) {

```

```

6     m[i] = malloc(cols * sizeof(int));    // her satirin verisi
7     if (!m[i]) {                          // hata: onceki satirlari temizle
8         for (size_t j = 0; j < i; j++) free(m[j]);
9         free(m);
10        return NULL;
11    }
12 }
13 return m;
14 }
15 void matrix_destroy(int **m, size_t rows) {
16     for (size_t i = 0; i < rows; i++) free(m[i]); // once satirlar
17     free(m);                                     // sonra isaretci dizisi
18 }

```

7. Yapılar (struct), Hizalama ve typedef

struct, farklı tipteki verileri tek bir birim altında toplar. İleri kullanımda hizalama (alignment) ve dolgu (padding) kritiktir.

7.1 Tanım, Erişim ve typedef

```

1 #include <stdio.h>
2 #include <string.h>
3
4 struct Point { int x, y; };           // etiketli yapı
5
6 typedef struct {                      // isimsiz + typedef
7     char name[32];
8     int age;
9     double salary;
10 } Employee;
11
12 int main(void) {
13     struct Point n = {3, 4};
14     printf("%d,%d\n", n.x, n.y);     // nokta erisimi
15
16     Employee c;
17     strcpy(c.name, "Ada");
18     c.age = 30; c.salary = 5000.0;
19
20     Employee *pc = &c;
21     printf("%s %d\n", pc->name, pc->age); // isaretci uzerinden -> operatoru
22     return 0;
23 }

```

Nokta vs Ok operatörü

yapi.alan → doğrudan erişim. isaretci->alan → işaretçi üzerinden erişim; (*isaretci).alan ile birebir aynıdır.

7.2 Bellek Hizalama ve Dolgu (Padding)

Derleyici, her üyeyi kendi boyutunun katı olan bir adrese yerleştirir (hizalama). Bu yüzden yapının boyutu, üyelerin toplamından *büyük* olabilir — aradaki boşluğa dolgu (padding) denir.

```

1 #include <stdio.h>
2 struct Bad {           // kotu siralama -> cok dolgu
3     char a;           // 1 bayt + 3 dolgu
4     int b;            // 4 bayt
5     char c;           // 1 bayt + 3 dolgu
6 };                    // toplam: 12 bayt
7
8 struct Good {          // iyi siralama (buyukten kucuge) -> az dolgu
9     int b;            // 4 bayt
10    char a;           // 1 bayt
11    char c;           // 1 bayt + 2 dolgu
12 };                    // toplam: 8 bayt
13
14 int main(void) {
15     printf("Kotu: %zu, Iyi: %zu\n", sizeof(struct Bad), sizeof(struct Good));
16     return 0;         // Kotu: 12, Iyi: 8
17 }

```

İpucu

Bellekten tasarruf için yapı üyelerini *büyükten küçüğe* sırala. Yapıyı sıkıştırmak için `#pragma pack(1)` veya `__attribute__((packed))` kullanılabilir — ama hizasız erişim bazı mimarilerde yavaşlar/çöker; ağ protokolleri için dikkatle kullan.

7.3 Kendine Referanslı Yapılar

Bir yapı, kendi tipinden bir *işaretçi* içerebilir. Bağlı liste, ağaç gibi veri yapılarının temelidir.

```

1 struct Node {
2     int data;
3     struct Node *next;    // kendine referans (isaretci olmalı!)
4 };

```

Dikkat

Yapı kendi tipini *değer* olarak içeremez (`struct Node next;`) — bu sonsuz boyut demektir. Yalnızca işaretçi (*) geçerlidir.

7.4 Esnek Dizi Üyesi (Flexible Array Member, C99)

Yapının sonundaki boyutsuz dizi, yapı + veriyi *tek* bellek bloğunda tutmayı sağlar.

```

1 #include <stdlib.h>
2 #include <string.h>
3 struct Text {
4     size_t length;
5     char data[];           // esnek dizi uyesi (boyutsuz, en sonda)
6 };
7 struct Text *text_create(const char *s) {
8     size_t n = strlen(s);
9     struct Text *m = malloc(sizeof(struct Text) + n + 1); // yapı + veri
10    if (!m) return NULL;
11    m->length = n;
12    strcpy(m->data, s);
13    return m;               // tek free ile hepsi temizlenir
14 }

```


8. Birlikler (union) ve Enum

8.1 union: Aynı Belleği Paylaşan Üyeler

Bir union, tüm üyelerini *aynı bellek adresinde* tutar. Boyutu, en büyük üyesi kadardır. Aynı anda yalnızca *bir* üye anlamlıdır.

```

1 #include <stdio.h>
2 union Value {
3     int    i;
4     float  f;
5     char   bytes[4];
6 };
7 int main(void) {
8     union Value d;
9     d.i = 1;                // şimdi 'i' anlamlı
10    printf("i = %d\n", d.i); // 1
11    d.f = 3.14f;             // aynı belleğe yazdı; 'i' artık geçersiz
12    printf("f = %g\n", d.f); // 3.14
13    printf("boyut = %zu\n", sizeof(union Value)); // 4 (en büyük üye)
14    return 0;
15 }
```

8.2 Type Punning: Bir Float'ın Bitlerini Görme

Union, bir değeri farklı tip gözüyle okumaya (type punning) izin verir — IEEE 754 float'ın ham bitlerini incelemek için kullanışlıdır.

```

1 #include <stdio.h>
2 #include <stdint.h>
3 int main(void) {
4     union { float f; uint32_t u; } x;
5     x.f = 1.0f;
6     printf("1.0f'in bitleri: 0x%08X\n", x.u); // 0x3F800000
7     return 0;
8 }
```

Bilgi

Endianness'i (bayt sırası) test etmenin klasik yolu da union'dır: küçük-endian makinede `union{int i; char c;} x; x.i=1;` sonucunda `x.c == 1` olur.

8.3 Tagged Union (Etiketli Birleşim)

Union'ın hangi üyesinin geçerli olduğunu bilmek için genelde bir enum "etiket" ile birlikte, bir struct içinde kullanılır. Bu desen, C'de tip-güvenli değişken (variant) oluşturma'nın standart yoludur.

```

1 #include <stdio.h>
2 typedef enum { TYPE_INT, TYPE_FLOAT, TYPE_TEXT } Type;
3
4 typedef struct {
5     Type type;                // hangi üye geçerli?
6     union {
7         int    i;
8         float  f;
9         const char *s;
10    };
11 }
```

```

10     } u;
11 } Value;
12
13 void print_value(Value d) {
14     switch (d.type) {
15         case TYPE_INT:    printf("int: %d\n", d.u.i); break;
16         case TYPE_FLOAT:  printf("float: %g\n", d.u.f); break;
17         case TYPE_TEXT:   printf("metin: %s\n", d.u.s); break;
18     }
19 }
20 int main(void) {
21     Value a = { .type = TYPE_INT, .u.i = 42 };
22     Value b = { .type = TYPE_TEXT, .u.s = "merhaba" };
23     print_value(a); print_value(b);
24     return 0;
25 }

```

8.4 enum: İsimli Sabitler

```

1 #include <stdio.h>
2 enum Color { RED, GREEN = 5, BLUE }; // 0, 5, 6
3
4 enum Day { MON = 1, TUE, WED, THU, FRI }; // 1,2,3,4,5
5
6 int main(void) {
7     enum Color r = BLUE;
8     printf("%d\n", r); // 6
9     printf("%d\n", FRI); // 5
10    return 0;
11 }

```

8.5 Bit Alanları (Bit Fields)

Yapı üyelerine bit düzeyinde boyut vererek belleği son derece verimli kullanmayı sağlar — donanım register'ları ve protokol başlıkları için ideal.

```

1 #include <stdio.h>
2 struct Flags {
3     unsigned active : 1; // 1 bit
4     unsigned hidden : 1; // 1 bit
5     unsigned priority : 3; // 3 bit (0-7)
6     unsigned          : 3; // 3 bit isimsiz dolgu
7 }; // toplam 1 bayta sigar
8 int main(void) {
9     struct Flags b = {0};
10    b.active = 1;
11    b.priority = 5;
12    printf("aktif=%u oncelik=%u boyut=%zu\n",
13          b.active, b.priority, sizeof(b)); // aktif=1 oncelik=5 boyut=4
14    return 0;
15 }

```

8.6 Bit İşlemleri ve Bayrak Maskeleri

Bit alanlarına alternatif olarak, bayrakları tek bir tamsayıda maskelerle de saklayabilirsin — taşınabilir ve hızlıdır.

```

1 #include <stdio.h>
2 #define FLAG_READ   (1u << 0)   // 0001
3 #define FLAG_WRITE  (1u << 1)   // 0010
4 #define FLAG_EXEC   (1u << 2)   // 0100
5
6 int main(void) {
7     unsigned perm = 0;
8     perm |= FLAG_READ | FLAG_WRITE;   // bit ekle (set)
9     perm &= ~FLAG_WRITE;              // bit temizle (clear)
10    perm ^= FLAG_EXEC;                 // bit degistir (toggle)
11
12    if (perm & FLAG_READ)               // bit test et
13        printf("okuma izni var\n");
14    printf("izin = %u\n", perm);        // 5
15    return 0;
16 }

```

Op	Ad	Kullanım
&	AND	Bit test etme, maskeleme.
	OR	Bit ekleme (set).
^	XOR	Bit değiştirme (toggle), takas.
~	NOT	Tüm bitleri terse çevirme.
« »	Kaydırma	2'nin kuvvetiyle hızlı çarpma/bölme.

9. Fonksiyon İşaretçileri ve Callback'ler

Fonksiyonların da bir adresi vardır; bu adresi bir işaretçide tutup daha sonra çağırabiliriz. Bu, C'de *davranışı veri gibi geçirmenin* yoludur — callback'ler, dispatch tabloları ve jenerik algoritmaların temeli.

9.1 Söz Dizimi

```

1 // int alan, int döndüren bir fonksiyonu gösteren isaretci:
2 int (*fp)(int);
3
4 // DİKKAT parantezler: int *fp(int) -> "int* döndüren fonksiyon" (farklı!)

```

```

1 #include <stdio.h>
2 int square(int x) { return x * x; }
3 int cube(int x)   { return x * x * x; }
4
5 int main(void) {
6     int (*op)(int);           // fonksiyon isaretcisi
7
8     op = square;               // & opsiyonel: op = &square;
9     printf("%d\n", op(5));     // 25 (*op)(5) ile aynı
10
11    op = cube;

```

```

12     printf("%d\n", op(3));    // 27
13     return 0;
14 }

```

9.2 Callback: qsort ile Sıralama

Standart kütüphane qsort, karşılaştırmayı bir callback ile alır — böylece herhangi bir tipi, herhangi bir ölçüte göre sıralar.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // qsort'un istediği imza: iki void* alır, int dondurur
5  int ascending(const void *a, const void *b) {
6      int x = *(const int *)a, y = *(const int *)b;
7      return (x > y) - (x < y);    // tasmadan güvenli karşılaştırma
8  }
9  int descending(const void *a, const void *b) {
10     return ascending(b, a);    // sırayı ters çevir
11 }
12
13 int main(void) {
14     int a[] = {5, 2, 8, 1, 9, 3};
15     size_t n = sizeof(a) / sizeof(a[0]);
16
17     qsort(a, n, sizeof(int), ascending);
18     for (size_t i = 0; i < n; i++) printf("%d ", a[i]); // 1 2 3 5 8 9
19     printf("\n");
20
21     qsort(a, n, sizeof(int), descending);
22     for (size_t i = 0; i < n; i++) printf("%d ", a[i]); // 9 8 5 3 2 1
23     printf("\n");
24     return 0;
25 }

```

9.3 Fonksiyon İşaretçisi Dizisi (Dispatch Table)

Uzun switch/if zincirleri yerine, fonksiyon işaretçilerinden oluşan bir tablo ile O(1) yönlendirme yapılır.

```

1  #include <stdio.h>
2  double add(double a, double b)    { return a + b; }
3  double subtract(double a, double b) { return a - b; }
4  double multiply(double a, double b) { return a * b; }
5  double divide(double a, double b)  { return b ? a / b : 0; }
6
7  int main(void) {
8      // İşlem karakterini fonksiyona esleyen tablo
9      struct { char op; double (*fn)(double, double); } table[] = {
10         {'+', add}, {'-', subtract}, {'*', multiply}, {'/', divide},
11     };
12     char op = '*'; double a = 6, b = 7;
13     for (size_t i = 0; i < sizeof(table)/sizeof(table[0]); i++)
14         if (table[i].op == op) {
15             printf("%g\n", table[i].fn(a, b)); // 42
16             break;
17         }
18     return 0;

```

19 }

9.4 typedef ile Okunabilir Fonksiyon İşaretçisi

Fonksiyon işaretçisi tipleri hızla okunmaz hale gelir; typedef bunu temizler.

```

1 #include <stdio.h>
2 typedef int (*BinOp)(int, int); // artık 'BinOp' bir tip
3
4 int add(int a, int b) { return a + b; }
5
6 void apply(BinOp f, int x, int y) { // parametre olarak temiz
7     printf("%d\n", f(x, y));
8 }
9 int main(void) {
10     BinOp op = add; // okunaklı bildirim
11     apply(op, 3, 4); // 7
12     return 0;
13 }
```

İpucu

Fonksiyon işaretçileri jenerik programlamanın kalbidir: qsort, bsearch, pthread_create, sinyal handler'ları ve olay sistemleri hep callback alır.

10. Değişken Argümanlı Fonksiyonlar (Variadic)

printf gibi değişken sayıda argüman alan fonksiyonlar <stdarg.h> ile yazılır.

```

1 #include <stdio.h>
2 #include <stdarg.h>
3
4 // İlk parametre sayıyı verir; sonra o kadar int okunur.
5 int sum(int count, ...) {
6     va_list ap;
7     va_start(ap, count); // 'count'tan SONRAKI argümanlar
8     int total = 0;
9     for (int i = 0; i < count; i++)
10         total += va_arg(ap, int); // sırayla oku (tip belirt)
11     va_end(ap); // temizle
12     return total;
13 }
14 int main(void) {
15     printf("%d\n", sum(3, 10, 20, 30)); // 60
16     printf("%d\n", sum(5, 1, 2, 3, 4, 5)); // 15
17     return 0;
18 }
```

Dikkat

Variadic fonksiyonlar tip güvenli *değildir*: va_arg'a yanlış tip verersen tanımsız davranış olur. Argüman sayısını/tiplerini bir biçim dizesi (printf gibi) ya da bir sayaç ile belirtmelisin. Sonlandırıcı olarak NULL da kullanılabilir.

11. Önişlemci ve Makrolar

Önişlemci, derlemeden önce metinsel dönüşümler yapar. Güçlüdür ama tuzaklıdır.

11.1 Nesne ve Fonksiyon Benzeri Makrolar

```
1 #define PI 3.14159 // nesne benzeri
2 #define SQUARE(x) ((x) * (x)) // fonksiyon benzeri
3 #define MAX(a, b) ((a) > (b) ? (a) : (b))
```

Dikkat

Makro argümanlarını ve tüm ifadeyi *parantezle*. #define SQUARE(x) x*x yazarsan SQUARE(1+2) → 1+2*1+2 = 5 olur (9 değil). Ayrıca MAX(i++, j++) argümanı iki kez değerlendirir — yan etkili argüman verme.

11.2 Koşullu Derleme ve Include Guard

```
1 #ifndef HEADER_H // include guard: çift dahil etmeyi onler
2 #define HEADER_H
3 /* ... bildirimler ... */
4 #endif
5
6 #ifdef DEBUG // yalnızca -DDEBUG ile derlenince
7     #define LOG(msg) fprintf(stderr, "[LOG] %s\n", msg)
8 #else
9     #define LOG(msg) ((void)0) // hiçbir şey yapmaz
10 #endif
11
12 #if defined(__linux__)
13     /* Linux'a özgü kod */
14 #elif defined(_WIN32)
15     /* Windows'a özgü kod */
16 #endif
```

11.3 Stringize (#) ve Token Paste (##)

```
1 #include <stdio.h>
2 #define STR(x) #x // argümanı stringe çevirir
3 #define CONCAT(a, b) a##b // iki token'i birleştirir
4
5 int main(void) {
6     printf("%s\n", STR(merhaba dünya)); // "merhaba dünya"
7     int xy = 5;
8     printf("%d\n", CONCAT(x, y)); // xy -> 5
9     return 0;
10 }
```

11.4 Faydalı Hazır Makrolar

Makro	Değer
<code>__FILE__</code>	Kaynak dosya adı (string).
<code>__LINE__</code>	Bulunulan satır numarası.
<code>__func__</code>	İçinde bulunan fonksiyonun adı (C99).
<code>__DATE__</code> / <code>__TIME__</code>	Derleme tarihi/saati.

Hata ayıklama makrosu

Dosya, satır ve fonksiyon bilgisiyle log.

```

1 #include <stdio.h>
2 #define LOG_ERR(fmt, ...) \
3     fprintf(stderr, "%s:%d (%s): " fmt "\n", \
4         __FILE__, __LINE__, __func__, ##__VA_ARGS__)
5
6 int main(void) {
7     LOG_ERR("kod = %d", 42);    // main.c:8 (main): kod = 42
8     return 0;
9 }
```

11.5 X-Macros: Tek Kaynaktan Çoklu Üretim

Aynı listeyi hem enum hem string dizisi olarak üretmek gibi tekrarları, X-macro deseni tek bir yerden yönetir.

```

1 #include <stdio.h>
2 #define COLORS \
3     X(RED) \
4     X(GREEN) \
5     X(BLUE)
6
7 typedef enum {
8     #define X(name) name,
9     COLORS
10    #undef X
11 } Color;
12
13 static const char *color_name[] = {
14     #define X(name) #name,
15     COLORS
16     #undef X
17 };
18 int main(void) {
19     Color r = GREEN;
20     printf("%d -> %s\n", r, color_name[r]);    // 1 -> GREEN
21     return 0;
22 }
```

12. Jenerik Programlama Teknikleri

C'nin şablonları yoktur, ama void *, makrolar ve C11'in `_Generic` özelliğiyle jenerik kod yazılabilir.

12.1 void * ile Jenerik Kap

```

1 #include <stdio.h>
2 #include <string.h>
3 // Herhangi bir tipteki elemanı genel takas (swap):
4 void swap(void *a, void *b, size_t size) {
5     unsigned char tmp[size];           // VLA (C99)
6     memcpy(tmp, a, size);
7     memcpy(a, b, size);
8     memcpy(b, tmp, size);
9 }
10 int main(void) {
11     int x = 1, y = 2;
12     swap(&x, &y, sizeof(int));
13     printf("%d %d\n", x, y);           // 2 1
14
15     double p = 1.5, q = 3.5;
16     swap(&p, &q, sizeof(double));
17     printf("%g %g\n", p, q);           // 3.5 1.5
18     return 0;
19 }
```

12.2 _Generic ile Tipe Göre Seçim (C11)

```

1 #include <stdio.h>
2 // Argumanın tipine göre farklı isim üretir
3 #define TYPE_NAME(x) _Generic((x), \
4     int: "int", \
5     double: "double", \
6     char: "char", \
7     char *: "string", \
8     default: "bilinmeyen")
9
10 int main(void) {
11     printf("%s\n", TYPE_NAME(42));      // int
12     printf("%s\n", TYPE_NAME(3.14));    // double
13     printf("%s\n", TYPE_NAME("selam")); // string
14     return 0;
15 }
```

Bilgi

`_Generic` derleme anında çalışır ve tip-güvenli "aşırı yükleme" (overloading) benzeri davranış sağlar. Standart kütüphanedeki `<tgmath.h>` tam olarak bunu kullanır.

13. Uygulama: İşaretçilerle Veri Yapıları

Öğrenilenleri birleştiren en iyi alıştırma, veri yapılarını sıfırdan yazmaktır.

13.1 Tekli Bağlı Liste


```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct Node {
5      int data;
6      struct Node *next;
7  } Node;
8
9  // Basa ekle: O(1). Liste başı çift-isaretci ile guncellenir.
10 void push_front(Node **head, int data) {
11     Node *new_node = malloc(sizeof(Node));
12     if (!new_node) return;
13     new_node->data = data;
14     new_node->next = *head; // yeni dugum eski basi gosterir
15     *head = new_node;      // bas artik yeni dugum
16 }
17
18 void print_list(const Node *head) {
19     for (const Node *p = head; p; p = p->next)
20         printf("%d -> ", p->data);
21     printf("NULL\n");
22 }
23
24 // Tum listeyi serbest birak (bellek sızıntısını onle)
25 void destroy(Node *head) {
26     while (head) {
27         Node *next = head->next; // ONCE sonrakini sakla
28         free(head);             // sonra bu dugumu birak
29         head = next;
30     }
31 }
32
33 int main(void) {
34     Node *list = NULL;
35     push_front(&list, 3);
36     push_front(&list, 2);
37     push_front(&list, 1);
38     print_list(list); // 1 -> 2 -> 3 -> NULL
39     destroy(list);
40     return 0;
41 }

```

Dikkat

Düğümü free etmeden önce next'i sakla. Ters sıra yaparsan (önce free) aslı işaretçiye erişip çökeresin.

13.2 Dinamik Yığın (Stack) — realloc ile Büyüyen Dizi

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct {
5      int *data;
6      size_t count; // dolu eleman
7      size_t capacity; // ayrilmis eleman
8  } Stack;

```

```
9
10 int stack_push(Stack *s, int value) {
11     if (s->count == s->capacity) {                // dolduysa buyut
12         size_t new_cap = s->capacity ? s->capacity * 2 : 4;
13         int *grown = realloc(s->data, new_cap * sizeof(int));
14         if (!grown) return -1;                    // hata: eski gecerli
15         s->data = grown;
16         s->capacity = new_cap;
17     }
18     s->data[s->count++] = value;
19     return 0;
20 }
21 int stack_pop(Stack *s, int *out) {
22     if (s->count == 0) return -1;                // bos
23     *out = s->data[--s->count];
24     return 0;
25 }
26 int main(void) {
27     Stack s = {0};
28     for (int i = 1; i <= 6; i++) stack_push(&s, i * 10);
29     int v;
30     while (stack_pop(&s, &v) == 0) printf("%d ", v); // 60 50 40 30 20 10
31     printf("\n");
32     free(s.data);
33     return 0;
34 }
```

İpucu

Bu "büyüyen dizi" (dynamic array / vector) deseni — kapasiteyi ikiye katlama — amortize edilmiş $O(1)$ ekleme sağlar ve gerçek dünyada en sık kullanılan veri yapısıdır.

KISIM 2

Linux Sistem Programlama

14. Sistem Çağrıları, Kernel ve errno

Sistem programlama, programın işletim sistemi çekirdeğiyle (kernel) doğrudan konuşmasıdır: dosya açmak, süreç oluşturmak, ağ üzerinden veri göndermek. Bu işlemler *sistem çağrıları* (system calls) ile yapılır.

14.1 Kullanıcı Alanı ve Çekirdek Alanı

Program normalde *kullanıcı alanında* (user space) çalışır; donanıma doğrudan erişemez. Bir kaynağa (dosya, ağ, bellek) ihtiyaç duyduğunda bir sistem çağrısı yaparak *çekirdek alanına* (kernel space) geçer. Çekirdek işi güvenli biçimde yapıp sonucu döndürür.

Kütüphane Fonksiyonu (libc)	Sistem Çağrısı (kernel)
fopen, fread, printf (tamponlu)	open, read, write (tamponsuz)
malloc	brk, sbrk, mmap
system	fork + execve + wait

Bilgi

man 2 <ad> bir *sistem çağrısını* anlatır (bölüm 2), man 3 <ad> ise bir *kütüphane fonksiyonunu* (bölüm 3). Örnek: man 2 open, man 3 printf.

14.2 Hata Yönetimi: errno

Çoğu sistem çağrısı başarısızlıkta -1 (ya da NULL) döndürür ve küresel errno değişkenine hata kodunu yazar. perror ve strerror bunu okunur mesaja çevirir.

```

1 #include <stdio.h>
2 #include <string.h>
3 #include <errno.h>
4 #include <fcntl.h>
5 #include <unistd.h>
6
7 int main(void) {
8     int fd = open("/olmayan/dosya", O_RDONLY);
9     if (fd == -1) {
10         // hata olustu
    
```

```

10 // errno'yu hemen kullan (sonraki cagrı ustune yazabilir)
11 fprintf(stderr, "Hata kodu: %d\n", errno);
12 perror("open"); // "open: No such file or directory"
13 fprintf(stderr, "strerror: %s\n", strerror(errno));
14 return 1;
15 }
16 close(fd);
17 return 0;
18 }

```

Dikkat

errno yalnızca bir çağrı başarısız olduğunda anlamlıdır; başarıda sıfırlanmaz. Bu yüzden önce dönüş değerini kontrol et, sonra errno'ya bak. Ayrıca errno'yu araya başka bir çağrı girmeden hemen kullan.

15. Düşük Seviye Dosya G/Ç

<stdio.h>'nin tamponlu FILE* akışlarının altında, çekirdeğin *dosya betimleyicileri* (file descriptor — fd) yatar. Bunlar küçük tamsayılardır: 0 = stdin, 1 = stdout, 2 = stderr.

15.1 open, read, write, close

Çağrı	Görev
open(yol, bayrak, mod)	Dosya açar/oluşturur; fd döndürür.
read(fd, tampon, n)	En çok n bayt okur; okunan sayıyı döndürür.
write(fd, tampon, n)	En çok n bayt yazar; yazılan sayıyı döndürür.
lseek(fd, offset, nereden)	Dosya konumunu değiştirir.
close(fd)	Betimleyiciyi kapatır.

open Bayrakları

Bayrak	Anlam
O_RDONLY / O_WRONLY / O_RDWR	Salt oku / salt yaz / oku-yaz.
O_CREAT	Yoksa oluştur (3. argüman: izin modu, örn. 0644).
O_TRUNC	Varsa içeriği sil (sıfırla).
O_APPEND	Sona ekle (her yazma dosya sonuna).
O_EXCL	O_CREAT ile: dosya zaten varsa hata ver.

Dosya kopyalama (cp'nin özü)

Bir dosyayı bayt bayt okuyup diğerine yazan tam program. Kısmi okuma/yazmaya ve sinyal kesintilerine dikkat edilir.

```

1  #include <fcntl.h>
2  #include <unistd.h>
3  #include <stdio.h>
4  #include <errno.h>
5
6  int main(int argc, char *argv[]) {
7      if (argc != 3) {
8          fprintf(stderr, "Kullanim: %s kaynak hedef\n", argv[0]);
9          return 1;
10     }
11     int src = open(argv[1], O_RDONLY);
12     if (src == -1) { perror("open kaynak"); return 1; }
13
14     // 0644 = kullanıcı oku/yaz, grup+digerleri oku
15     int dst = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
16     if (dst == -1) { perror("open hedef"); close(src); return 1; }
17
18     char buffer[4096];
19     ssize_t n_read;
20     while ((n_read = read(src, buffer, sizeof(buffer))) > 0) {
21         ssize_t n_written = 0;
22         while (n_written < n_read) { // kısmi yazmayı ele al
23             ssize_t r = write(dst, buffer + n_written, n_read - n_written);
24             if (r == -1) {
25                 if (errno == EINTR) continue; // sinyal kesti; tekrar dene
26                 perror("write"); close(src); close(dst); return 1;
27             }
28             n_written += r;
29         }
30     }
31     if (n_read == -1) perror("read");
32
33     close(src);
34     close(dst);
35     return 0;
36 }

```

Dikkat

read/write istediğin kadar bayt işlemeyebilir (kısmi transfer). write özellikle boru ve sokette daha az yazabilir. Bu yüzden *her zaman* döngüyle, dönüş değerini kontrol ederek yaz. EINTR (sinyalle kesilme) durumunda çağırışı tekrarla.

15.2 lseek ve Dosya Konumu

```

1  #include <fcntl.h>
2  #include <unistd.h>
3  #include <stdio.h>
4  int main(void) {
5      int fd = open("veri.bin", O_RDWR | O_CREAT, 0644);
6
7      write(fd, "MERHABA", 7);
8      lseek(fd, 0, SEEK_SET); // basa don
9      char buf[8] = {0};
10     read(fd, buf, 7);
11     printf("%s\n", buf); // MERHABA
12 }

```

```

13 // Seyrek dosya (sparse file): ileri atla, delik oluşur
14 lseek(fd, 1000, SEEK_END); // sondan 1000 bayt ileri
15 write(fd, "SON", 3);
16
17 off_t size = lseek(fd, 0, SEEK_END); // dosya boyutu
18 printf("boyut = %ld\n", (long)size);
19
20 close(fd);
21 return 0;
22 }

```

SEEK sabitleri

SEEK_SET = dosya başından, SEEK_CUR = mevcut konumdan, SEEK_END = dosya sonundan itibaren ofset.

15.3 dup, dup2 ve Betimleyici Yönlendirme

dup2 bir betimleyiciyi başka bir numaraya kopyalar — kabuğun > yönlendirmesinin ve boru bağlamanın temelidir.

```

1 #include <fcntl.h>
2 #include <unistd.h>
3 int main(void) {
4     int fd = open("cikti.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
5     dup2(fd, STDOUT_FILENO); // stdout artık dosyaya gider
6     close(fd);
7
8     // Bu andan sonra tum stdout ciktileri cikti.txt'ye yazilir
9     write(STDOUT_FILENO, "Bu satir dosyaya gitti\n", 23);
10    return 0;
11 }

```

15.4 Betimleyici mi, Akış mı?

fd (open/read/write)	FILE* (fopen/fread)
Ham sistem çağrısı, tamponsuz	Kullanıcı alanında tamponlu (hızlı)
Küçük tamsayı	FILE yapısına işaretçi
Sokete/boruya doğrudan uyar	Satır/tam tamponlama; fflush gerekir

16. İleri Dosya G/Ç

Temel open/read/write üzerine, gerçek uygulamaların ihtiyaç duyduğu konumsal G/Ç, dağınık G/Ç, dosya kilitleme, kalıcılık (durability), geçici dosyalar ve verimli kopyalama teknikleri.

16.1 Konumsal G/Ç: pread ve pwrite

pread/pwrite, dosya konumunu (offset) *değiştirmeden* belirli bir konumdan okur/yazar. lseek + read'ın aksine atomiktir — iş parçacıklarının aynı dosyayı yarışmadan kullanmasını sağlar.

```

1 #include <fcntl.h>
2 #include <unistd.h>
3 #include <stdio.h>
4 int main(void) {
5     int fd = open("veri.bin", O_RDWR | O_CREAT, 0644);
6
7     char record[16] = "KAYIT-0000000000";
8     pwrite(fd, record, 16, 0);          // 0. konuma yaz (offset degismez)
9     pwrite(fd, record, 16, 1600);      // 100. kaydın yerine yaz
10
11     char buffer[16];
12     pread(fd, buffer, 16, 1600);      // dosya konumundan BAGIMSIZ oku
13     printf("%.16s\n", buffer);
14
15     close(fd);
16     return 0;
17 }

```

Bilgi

Çok iş parçacıklı bir programda tek bir açık dosyaya paralel erişim için pread/pwrite şarttır: dosya konumu paylaşıldığından lseek+read ikilisi yarış durumu yaratır.

16.2 Dağınık/Toplu G/Ç: readv ve writev (Scatter-Gather)

Tek sistem çağrısıyla birden çok tampona okur / birden çok tampondan yazar. Başlık + gövdeyi ayrı tamponlardan tek atomik yazmayla göndermek için idealdir.

```

1 #include <sys/uio.h>
2 #include <unistd.h>
3 #include <string.h>
4 int main(void) {
5     const char *header = "[BASLIK] ";
6     const char *body   = "mesaj govdesi\n";
7
8     struct iovec parts[2];
9     parts[0].iov_base = (void *)header;
10    parts[0].iov_len  = strlen(header);
11    parts[1].iov_base = (void *)body;
12    parts[1].iov_len  = strlen(body);
13
14    // İki tampon TEK writev çağrısıyla, sırayla, atomik yazılır
15    writev(STDOUT_FILENO, parts, 2);    // [BASLIK] mesaj govdesi
16    return 0;
17 }

```

16.3 Dosya Bayraklarını Yönetme: fcntl

fcntl, açık bir betimleyicinin özelliklerini çalışma anında değiştirir: non-blocking moda geçme, FD_CLOEXEC (exec'te otomatik kapanma) gibi.

```

1 #include <fcntl.h>
2 #include <unistd.h>
3 int main(void) {
4     int fd = open("dosya.txt", O_RDONLY);
5
6     // Mevcut bayrakları al, O_NONBLOCK ekle, geri yaz

```

```

7   int flags = fcntl(fd, F_GETFL);
8   fcntl(fd, F_SETFL, flags | O_NONBLOCK);      // artık bloklamaz
9
10  // exec sonrası otomatik kapansin (betimleyici sizintisini onler)
11  int fd_flags = fcntl(fd, F_GETFD);
12  fcntl(fd, F_SETFD, fd_flags | FD_CLOEXEC);
13
14  close(fd);
15  return 0;
16 }

```

Dikkat

Betimleyiciler fork sonrası çocukta açık kalır ve exec sonrası da (varsayılan) açık kalır — bu bir güvenlik/kaynak sızıntısıdır. Hassas dosyaları FD_CLOEXEC ile işaretle ya da open'da O_CLOEXEC bayrağını kullan.

16.4 Dosya Kilitleme (fcntl F_SETLK)

Birden çok süreç aynı dosyaya yazarken tutarlılık için öneri kilitleri (advisory locks) kullanılır.

```

1  #include <fcntl.h>
2  #include <unistd.h>
3  #include <stdio.h>
4  int main(void) {
5      int fd = open("kayit.db", O_RDWR | O_CREAT, 0644);
6
7      struct flock lock = {0};
8      lock.l_type = F_WRLCK;      // yazma kilidi (F_RDLCK: okuma, F_UNLCK: ac)
9      lock.l_whence = SEEK_SET;
10     lock.l_start = 0;           // offsetten...
11     lock.l_len = 0;             // ...sona kadar (0 = tum dosya)
12
13     if (fcntl(fd, F_SETLKW, &lock) == -1) { // F_SETLKW: kilit bosalana kadar bekle
14         perror("kilit"); return 1;
15     }
16     // --- kritik bolge: dosyaya guvenle yaz ---
17     pwrite(fd, "guncel", 6, 0);
18
19     lock.l_type = F_UNLCK;
20     fcntl(fd, F_SETLK, &lock); // kilidi birak
21     close(fd);
22     return 0;
23 }

```

F_SETLK vs F_SETLKW

F_SETLK kilit mesgulse hemen -1 döner (EAGAIN). F_SETLKW ise kilit serbest kalana kadar bekler (W = wait). Basit dosya kilidi için flock(fd, LOCK_EX) da kullanılabilir.

16.5 Kalıcılık: fsync, fdatasync

write verinin diske yazıldığını garanti etmez — çekirdek onu önbellekte tutar. Veritabanları ve dosya bütünlüğü için diske zorlamak gerekir.

```

1  #include <fcntl.h>
2  #include <unistd.h>

```



```

3 int main(void) {
4     int fd = open("onemli.log", O_WRONLY | O_CREAT | O_APPEND, 0644);
5     write(fd, "kritik islem tamam\n", 19);
6     fsync(fd);           // veri + metadata'yi diske zorla (guc kesintisine dayanikli)
7     // fdatsync(fd); // yalnızca veri (boyut degismediye daha hizli)
8     close(fd);
9     return 0;
10 }

```

16.6 Güvenli Geçici Dosya: mkstemp

```

1 #include <stdlib.h>
2 #include <unistd.h>
3 #include <stdio.h>
4 int main(void) {
5     char template[] = "/tmp/uygXXXXXX"; // son 6 karakter X olmalı
6     int fd = mkstemp(template);           // benzersiz ad üretir + açar (güvenli)
7     if (fd == -1) { perror("mkstemp"); return 1; }
8     printf("Gecici dosya: %s\n", template);
9
10    write(fd, "gecici veri", 11);
11    // Genellikle isim hemen silinir; fd acik kaldigi surece dosya yasar
12    unlink(template);                       // program bitince tamamen yok olur
13    close(fd);
14    return 0;
15 }

```

Dikkat

Asla tmpnam/mktemp kullanma — ad üretimiyle açma arasında yarış (TOCTOU) güvenlik açığı vardır. mkstemp adı üretir ve atomik olarak açar.

16.7 Verimli Kopyalama: sendfile

sendfile, veriyi kullanıcı alanına taşımadan çekirdek içinde bir fd'den diğerine kopyalar — dosya sunucularında (statik içerik) çok hızlıdır.

```

1 #include <sys/sendfile.h>
2 #include <sys/stat.h>
3 #include <fcntl.h>
4 #include <unistd.h>
5 int main(int argc, char *argv[]) {
6     int in_fd = open(argv[1], O_RDONLY);
7     int out_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
8     struct stat st; fstat(in_fd, &st);
9
10    off_t offset = 0;
11    // Tum dosyayi cekirdek icinde kopyala (read/write dongusu YOK)
12    sendfile(out_fd, in_fd, &offset, st.st_size);
13
14    close(in_fd); close(out_fd);
15    return 0;
16 }

```

16.8 stdio Tamponlaması: setvbuf

FILE* akışları kullanıcı alanında tamponlanır. Tamponlama modunu kontrol etmek, performans ve etkileşimli çıktı için önemlidir.

Mod	Davranış
_IOFBF	Tam tamponlama: tampon dolunca yazılır (dosyalarda varsayılan).
_IOLBF	Satır tamponlama: \n görünce yazılır (terminalde varsayılan).
_IONBF	Tamponsuz: her yazma anında gider (stderr).

```

1 #include <stdio.h>
2 int main(void) {
3     // stdout'u tamponsuz yap (log çıktısı anında görünsün)
4     setvbuf(stdout, NULL, _IONBF, 0);
5     printf("Bu satır anında görünür"); // \n olmadan bile
6     // Alternatif: fflush(stdout); ile tamponu elle bosalt
7     return 0;
8 }

```

İpucu

Program çökerse tamponlanmış printf çıktısı *kaybolabilir*. Hata ayıklarken ya stderr kullan (tamponsuz), ya her kritik noktada fflush(stdout) çağır, ya da setvbuf ile tamponlamayı kapat.

17. Dosya Sistemi: stat, Dizinler, İzinler

17.1 stat ile Dosya Bilgisi

```

1 #include <sys/stat.h>
2 #include <stdio.h>
3 #include <time.h>
4 int main(int argc, char *argv[]) {
5     if (argc != 2) { fprintf(stderr, "Kullanım: %s dosya\n", argv[0]); return 1; }
6     struct stat st;
7     if (stat(argv[1], &st) == -1) { perror("stat"); return 1; }
8
9     printf("Boyut      : %ld bayt\n", (long)st.st_size);
10    printf("Inode       : %ld\n", (long)st.st_ino);
11    printf("Link sayısı: %ld\n", (long)st.st_nlink);
12    printf("İzinler    : %o\n", st.st_mode & 0777);
13    printf("Sahip UID  : %d\n", st.st_uid);
14
15    if (S_ISREG(st.st_mode)) printf("Tur: normal dosya\n");
16    if (S_ISDIR(st.st_mode)) printf("Tur: dizin\n");
17    if (S_ISLNK(st.st_mode)) printf("Tur: sembolik bağlantı\n");
18
19    printf("Son değişim: %s", ctime(&st.st_mtime));
20    return 0;
21 }

```

Bilgi

stat sembolik bağlantıyı *takip eder* (hedefi anlatır); bağlantının kendisini incelemek için lstat, açık bir fd için ise fstat kullanılır.

17.2 Dizin Okuma (opendir/readdir)

```

1  #include <dirent.h>
2  #include <stdio.h>
3  int main(int argc, char *argv[]) {
4      const char *path = (argc > 1) ? argv[1] : ".";
5      DIR *d = opendir(path);
6      if (!d) { perror("opendir"); return 1; }
7
8      struct dirent *entry;
9      while ((entry = readdir(d)) != NULL)    // her çağrı bir girdi
10         printf("%s\n", entry->d_name);      // . ve .. dahil
11
12     closedir(d);
13     return 0;
14 }
```

17.3 İzinler: chmod, access, umask

```

1  #include <sys/stat.h>
2  #include <unistd.h>
3  int main(void) {
4      chmod("betik.sh", 0755);    // rwxr-xr-x
5
6      if (access("dosya.txt", R_OK) == 0)    // okuma izni var mı?
7          /* okunabilir */;
8      if (access("dosya.txt", F_OK) == 0)    // dosya var mı?
9          /* mevcut */;
10     return 0;
11 }
```

Sekizlik izin gösterimi

İzinler üç haneli sekizlik sayıdır: sahip/grup/diğerleri. Her hane oku(4)+yaz(2)+çalış(1) toplamıdır. Örnek: 0644 = rw-r-r-, 0755 = rwxr-xr-x, 0600 = rw-----.

17.4 Dizin ve Bağlantı İşlemleri

Çağrı	Görev
<code>mkdir(yol, mod)</code>	Dizin oluştur.
<code>rmdir(yol)</code>	Boş dizini sil.
<code>unlink(yol)</code>	Dosyayı (bağlantıyı) sil.
<code>rename(eski, yeni)</code>	Yeniden adlandır/taşı (atomik).
<code>link(hedef, ad)</code>	Sabit bağlantı (hard link) oluştur.
<code>symlink(hedef, ad)</code>	Sembolik bağlantı oluştur.
<code>readlink(yol, buf, n)</code>	Sembolik bağlantının hedefini oku.
<code>chdir(yol) / getcwd(buf, n)</code>	Çalışma dizinini değiştir / öğren.

```

1 #include <sys/stat.h>
2 #include <unistd.h>
3 #include <stdio.h>
4 int main(void) {
5     mkdir("test_dizin", 0755);           // dizin olustur
6
7     int fd = open("test_dizin/a.txt", O_WRONLY | O_CREAT, 0644);
8     write(fd, "veri", 4); close(fd);
9
10    link("test_dizin/a.txt", "test_dizin/b.txt"); // sabit baglanti
11    symlink("a.txt", "test_dizin/c.txt");         // sembolik baglanti
12    rename("test_dizin/a.txt", "test_dizin/d.txt"); // yeniden adlandir
13
14    char cwd[256];
15    getcwd(cwd, sizeof cwd);                 // mevcut dizin
16    printf("Calisma dizini: %s\n", cwd);
17
18    // Temizlik
19    unlink("test_dizin/b.txt");
20    unlink("test_dizin/c.txt");
21    unlink("test_dizin/d.txt");
22    rmdir("test_dizin");                     // dizin bosalinca silinebilir
23    return 0;
24 }

```

Sabit vs Sembolik Bağlantı

Sabit bağlantı aynı inode'a ikinci bir isimdir; asıl silinse bile veri yaşar (link sayısı 0 olana dek). **Sembolik bağlantı** başka bir *yola* işaret eden küçük bir dosyadır; hedef silinirse "kırık" (dangling) olur.

17.5 Dizin Ağacında Gezinme (nftw)

`nftw`, bir dizin ağacını özyinelemeli olarak dolaşır her girdi için bir callback çağırır — `find` komutunun temelidir.

```

1 #define _XOPEN_SOURCE 500
2 #include <ftw.h>
3 #include <stdio.h>
4
5 // Her dosya/dizin icin cagrilir

```

```

6 int visit(const char *path, const struct stat *st, int type, struct FTW *f) {
7     (void)type;
8     printf("%s%s (%ld bayt)\n", f->level * 2, "", path, (long)st->st_size);
9     return 0;    // 0: devam et, sıfır dışı: durdur
10 }
11 int main(int argc, char *argv[]) {
12     const char *root = (argc > 1) ? argv[1] : ".";
13     // 20: aynı anda açık fd sınırı; FTW_PHYS: sembolik bağlantıları izleme
14     nftw(root, visit, 20, FTW_PHYS);
15     return 0;
16 }

```

18. G/Ç Çoğullama: select, poll, epoll

Tek bir iş parçacığının *birden çok* betimleyiciyi (soket, boru, terminal) aynı anda izlemesini sağlar: "hangisi okumaya hazır?" Ağ sunucularının kalbidir.

18.1 Neden Gerekli?

Bloklayan read tek bir fd'de bekler; o gelene kadar diğerleri işlenemez. G/Ç çoğullama, "bu fd'lerden herhangi biri hazır olunca beni uyandır" der — böylece tek iş parçacığı binlerce bağlantıyı yönetir.

18.2 select

```

1 #include <sys/select.h>
2 #include <unistd.h>
3 #include <stdio.h>
4 int main(void) {
5     fd_set readfds;
6     struct timeval timeout = { .tv_sec = 5, .tv_usec = 0 };
7
8     FD_ZERO(&readfds);           // kumeyi temizle
9     FD_SET(STDIN_FILENO, &readfds); // stdin'i izle
10
11     // 1. arg: en büyük fd + 1. Doner: hazır fd sayısı (0 = zaman asimi)
12     int ready = select(STDIN_FILENO + 1, &readfds, NULL, NULL, &timeout);
13
14     if (ready == -1) perror("select");
15     else if (ready == 0) printf("5 saniye içinde giriş yok.\n");
16     else if (FD_ISSET(STDIN_FILENO, &readfds)) {
17         char buf[128];
18         ssize_t n = read(STDIN_FILENO, buf, sizeof buf - 1);
19         if (n > 0) { buf[n] = '\0'; printf("Girdi: %s", buf); }
20     }
21     return 0;
22 }

```

Dikkat

select'in sınırları vardır: fd sayısı FD_SETSIZE (genelde 1024) ile sınırlıdır ve her çağrıda küme yeniden kurulmalıdır. Çok sayıda bağlantı için poll ya da epoll tercih edilir.

18.3 poll

```

1 #include <poll.h>
2 #include <unistd.h>
3 #include <stdio.h>
4 int main(void) {
5     struct pollfd fds[1];
6     fds[0].fd = STDIN_FILENO;
7     fds[0].events = POLLIN;           // okumaya hazır olmasını izle
8
9     int ready = poll(fds, 1, 5000); // 5000 ms zaman asımı
10    if (ready > 0 && (fds[0].revents & POLLIN)) {
11        char buf[128];
12        ssize_t n = read(STDIN_FILENO, buf, sizeof buf - 1);
13        if (n > 0) { buf[n] = '\0'; printf("Girdi: %s", buf); }
14    } else if (ready == 0) {
15        printf("Zaman asımı.\n");
16    }
17    return 0;
18 }

```

18.4 epoll — Ölçeklenebilir Linux Çözümü

epoll, on binlerce bağlantıyı verimli izler: her çağrıda tüm listeyi taramaz, yalnızca *hazır olanları* döndürür. Yüksek performanslı Linux sunucularının (nginx, redis) temelidir.

```

1 #include <sys/epoll.h>
2 #include <unistd.h>
3 #include <stdio.h>
4 int main(void) {
5     int ep = epoll_create1(0);           // epoll ornegi olustur
6
7     struct epoll_event event = {0};
8     event.events = EPOLLIN;             // okuma olaylarini izle
9     event.data.fd = STDIN_FILENO;
10    epoll_ctl(ep, EPOLL_CTL_ADD, STDIN_FILENO, &event); // fd ekle
11
12    struct epoll_event events[10];
13    printf("Giris bekleniyor...\n");
14
15    // -1: sonsuza kadar bekle. Doner: hazır olay sayısı
16    int n = epoll_wait(ep, events, 10, -1);
17    for (int i = 0; i < n; i++) {
18        if (events[i].data.fd == STDIN_FILENO) {
19            char buf[128];
20            ssize_t r = read(STDIN_FILENO, buf, sizeof buf - 1);
21            if (r > 0) { buf[r] = '\0'; printf("Girdi: %s", buf); }
22        }
23    }
24    close(ep);
25    return 0;
26 }

```

API	Ölçek	Not
select	Düşük (< 1024)	Taşınabilir (POSIX); küme her çağrıda kurulur.
poll	Orta	Taşınabilir; fd limiti yok ama O(n) tarama.
epoll	Yüksek (10K+)	Linux'a özgü; O(1), olay tetiklemeli.

İpucu

epoll iki modda çalışır: **level-triggered** (varsayılan; veri durdukça uyarır) ve **edge-triggered** (EPOLLET; yalnızca durum *değişince* uyarır). Edge-triggered daha hızlıdır ama tüm veriyi non-blocking döngüde okumayı gerektirir.

19. Süreçler: fork, exec, wait

Süreç (process), çalışan bir programdır. Linux'ta her süreç fork ile oluşturulur ve exec ile başka bir programa dönüşebilir.

19.1 fork: Süreç Çoğaltma

fork çağırın süreci ikiye böler: ebeveyn (parent) ve neredeyse özdeş bir çocuk (child). Tek çağrı, *iki kez* döner.

```

1 #include <unistd.h>
2 #include <stdio.h>
3 #include <sys/wait.h>
4 int main(void) {
5     printf("fork oncesi (tek surec)\n");
6
7     pid_t pid = fork();          // burada surec ikiye bolunur
8
9     if (pid < 0) {                // hata
10         perror("fork");
11         return 1;
12     } else if (pid == 0) {        // COÇUK: fork 0 dondurur
13         printf("Cocuk : PID=%d, ebeveyn=%d\n", getpid(), getppid());
14     } else {                     // EBEVEYN: fork cocugun PID'sini dondurur
15         printf("Ebeveyn: PID=%d, cocuk=%d\n", getpid(), pid);
16         wait(NULL);              // cocugun bitmesini bekle (zombi olmasın)
17     }
18     return 0;
19 }
```

fork dönüşü	Anlam
< 0	Hata (fork başarısız).
== 0	Çocuk sürecindeyiz.
> 0	Ebeveyndeyiz; değer çocuğun PID'sidir.

Bilgi

Çocuk, ebeveynin belleğinin bir *kopyasını* alır (copy-on-write: gerçek kopya ancak biri yazınca yapılır). Açık dosya betimleyicileri de paylaşılır.

19.2 exec: Program Değiştirme

exec ailesi, mevcut sürecin bellek imajını *yenı bir programla* tümüyle değiştirir. Başarılıysa geri dönmez.

Fonksiyon**Özellik (l=liste, v=vektör, p=PATH, e=env)**

execl	Argümanlar liste olarak, tam yol.
execlp	Liste + PATH'te arar.
execv	Argümanlar dizi (vektör) olarak.
execvp	Vektör + PATH'te arar (en sık kullanılan).
execve	Vektör + özel ortam; <i>gerçek</i> sistem çağırısı.

fork + exec: kabuğun temel döngüsü

Bir komut çalıştırmanın klasik kalıbı: fork ile çocuk üret, çocukta exec ile komuta dönüş, ebeveynde bekle.

```

1 #include <unistd.h>
2 #include <sys/wait.h>
3 #include <stdio.h>
4 int main(void) {
5     pid_t pid = fork();
6     if (pid == 0) {
7         // COÇUK: "ls -l" komutuna dönüş
8         char *argv[] = {"ls", "-l", NULL}; // NULL ile bitmeli
9         execvp("ls", argv);
10        // execvp başarılıysa BURAYA ASLA GELİNMEZ
11        perror("execvp"); // buraya gelinirse hata var
12        _exit(127); // exec başarısız
13    } else if (pid > 0) {
14        int status;
15        waitpid(pid, &status, 0); // cocuğu bekle
16        if (WIFEXITED(status))
17            printf("Cocuk %d çıktı, kod=%d\n", pid, WEXITSTATUS(status));
18    }
19    return 0;
20 }
```

19.3 wait, waitpid ve Çıkış Durumu

Ebeveyn, çocuğun çıkış durumunu wait/waitpid ile toplamalıdır. Aksi halde çocuk "zombi" olarak süreç tablosunda kalır.

Makro	Anlam
WIFEXITED(s)	Normal mi çıktı (return/exit)?
WEXITSTATUS(s)	Çıkış kodu (0–255).
WIFSIGNALED(s)	Sinyalle mi öldü?
WTERMSIG(s)	Öldüren sinyal numarası.

Dikkat

Zombi süreç: çocuk bitmiş ama ebeveyn wait etmemiştir; çıkış durumu için tabloda yer tutar.
Öksüz (orphan) süreç: ebeveyn önce ölmüştür; çocuk init/systemd (PID 1) tarafından evlat edinilir. Zombileri önlemek için her fork’a bir wait düşür ya da SIGCHLD handler’ı kur.

20. İleri Süreç Yönetimi

fork/exec/wait temellerinin ötesinde: exec ailesinin tümü, ortam değişkenleri, çıkış kancaları, süreç grupları/oturumlar, arka plan servisleri (daemon) ve kaynak sınırları.

20.1 exec Ailesinin Tümü

Altı exec varyantı, argümanları nasıl aldıklarına ve PATH/ortam kullanıp kullanmadıklarına göre farklılaşır. Sonek harfleri: **l**=liste, **v**=vektör, **p**=PATH’te ara, **e**=özel ortam.

```

1 #include <unistd.h>
2 int main(void) {
3     // l: argümanlar tek tek, NULL ile biter, TAM YOL
4     execl("/bin/ls", "ls", "-l", "/tmp", (char *)NULL);
5
6     // v: argümanlar dizi olarak
7     char *av[] = {"ls", "-l", NULL};
8     execv("/bin/ls", av);
9
10    // p: komutu PATH’te arar (tam yol gerekmez) -- en pratik
11    execlp("ls", "ls", "-l", (char *)NULL);
12    execvp("ls", av);
13
14    // e: çağırana özel ortam (environment) verir
15    char *env[] = {"YOL=/özel", "DIL=tr", NULL};
16    execl("/bin/prog", "prog", (char *)NULL, env);
17    execvpe("prog", av, env); // GNU eklentisi
18
19    return 0; // exec başarılı olursa buraya asla gelinmez
20 }
```

Argüman listesi ve program adı

exec’e verilen ilk argüman, çalışacak programın argv[0]’ı olur — genelde program adıdır ama farklı da olabilir. Liste (1) biçiminde son argüman mutlaka (char *)NULL olmalıdır.

20.2 Ortam Değişkenleri

```

1 #include <stdlib.h>
2 #include <stdio.h>
3 extern char **environ;           // tum ortam: "AD=deger" dizisi (NULL biter)
4
5 int main(void) {
6     printf("HOME = %s\n", getenv("HOME")); // oku
7
8     setenv("UYGULAMA_MODU", "gelistirme", 1); // yaz (1 = ustune yaz)
9     printf("MOD = %s\n", getenv("UYGULAMA_MODU"));
10
11     unsetenv("UYGULAMA_MODU");      // sil
12
13     // Tum ortami dolas
14     for (char **e = environ; *e; e++)
15         printf("%s\n", *e);
16     return 0;
17 }

```

20.3 Çıkış Kancaları: atexit ve _exit

```

1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 void cleanup1(void) { printf("temizlik 1\n"); }
6 void cleanup2(void) { printf("temizlik 2\n"); }
7
8 int main(void) {
9     atexit(cleanup1);           // exit() ya da main return'de calisir
10    atexit(cleanup2);           // TERS sirada: once cleanup2, sonra cleanup1
11
12    printf("is bitti\n");
13    return 0;                   // exit(0) cagrilir -> kancalar tetiklenir
14    // _exit(0);                 // FARK: kancalari CALISTIRMAZ, tampon bosaltmaz
15 }

```

Dikkat

fork edilen çocukta hata olursa, exit yerine _exit (ya da _Exit) kullan. exit, ebeveynden miras kalan stdio tamponlarını boşaltarak çıktının *iki* kez yazılmasına yol açabilir.

20.4 Süreç Grupları ve Oturumlar

Her süreç bir *süreç grubuna*, her grup bir *oturuma* (session) aittir. Kabuk, iş kontrolü (job control) ve sinyal dağıtımını bunlarla çalışır; bir terminale bağlı tüm süreçlere aynı anda sinyal (örn. Ctrl+C) gönderilir.

```

1 #include <unistd.h>
2 #include <stdio.h>
3 int main(void) {
4     printf("PID=%d PGID=%d SID=%d\n",
5           getpid(), getpgid(), getsid(0));
6
7     setpgid(0, 0);           // kendini yeni bir surec grubu lideri yap
8     return 0;
9 }

```

20.5 Arka Plan Servisi (Daemon) Oluşturma

Bir daemon, terminalden kopmuş, arka planda çalışan bir servistir. Klasik "double-fork" tekniğiyle oluşturulur.

```

1 #include <unistd.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
4 #include <sys/stat.h>
5
6 void daemonize(void) {
7     // 1) İlk fork: ebeveyn çıkar, çocuk devam eder (kabuk komut istemine doner)
8     if (fork() > 0) exit(0);
9
10    // 2) Yeni oturum lideri ol: kontrol terminalinden kopar
11    setsid();
12
13    // 3) İkinci fork: terminal edinmeyi kesin olarak engelle
14    if (fork() > 0) exit(0);
15
16    umask(0);                // dosya izin maskesini sıfırla
17    chdir("/");              // koke gec (bagli dizin unmount edilebilsin)
18
19    // 4) Standart betimleyicileri /dev/null'a yönlendir
20    int null = open("/dev/null", O_RDWR);
21    dup2(null, STDIN_FILENO);
22    dup2(null, STDOUT_FILENO);
23    dup2(null, STDERR_FILENO);
24    if (null > 2) close(null);
25    // Artık arka planda çalışan bir servisiz.
26 }

```

İpucu

Modern sistemlerde daemon'ları elle yazmak yerine systemd servis birimleri tercih edilir; systemd fork/setsid/log yönlendirmeyi kendi halleder. Yine de tekniği bilmek altyapıyı anlamak için değerlidir.

20.6 Kaynak Sınırları: getrlimit / setrlimit

```

1 #include <sys/resource.h>
2 #include <stdio.h>
3 int main(void) {
4     struct rlimit lim;
5     getrlimit(RLIMIT_NOFILE, &lim);    // açık dosya sınırı
6     printf("Açık dosya: yumusak=%ld sert=%ld\n",
7           (long)lim.rlim_cur, (long)lim.rlim_max);
8
9     lim.rlim_cur = 256;                  // yumusak sınırı düşür
10    setrlimit(RLIMIT_NOFILE, &lim);
11
12    // RLIMIT_CPU (cpu saniyesi), RLIMIT_AS (adres alanı),
13    // RLIMIT_CORE (core dump boyutu) de ayarlanabilir.
14    return 0;
15 }

```

21. Terminal Komutları ve Program Çalıştırma

Bir C programından kabuk komutu ya da başka bir program çalıştırmanın çeşitli yolları: en basitten (system) en güçlüye (elle fork/exec + boru ile çıktı yakalama) kadar.

21.1 system: En Basit Yol (ve Tehlikeleri)

system, komutu /bin/sh -c ile çalıştırır ve bitmesini bekler. Basittir ama kabuk enjeksiyonuna (shell injection) açıktır.

```

1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <sys/wait.h>
4 int main(void) {
5     int status = system("ls -l /tmp | wc -l"); // kabuk ozelliklerini kullanir
6
7     if (status == -1) perror("system");
8     else printf("Cikis kodu: %d\n", WEXITSTATUS(status));
9
10    return 0;
11 }
```

Dikkat

Kullanıcı girdisini asla doğrudan system'e verme: system(strcat("rm ", kullanıcı_girdisi)) felakettir — kullanıcı "; rm -rf /" yazarsa çalışır. Güvenlik gerektiğinde fork + execvp kullan (kabuk araya girmez, argümanlar yorumlanmaz).

21.2 popen: Komut Çıktısını Okuma

popen, komutu çalıştırıp çıktısını (ya da girdisini) bir FILE* akışı olarak verir — komutun sonucunu satır satır okumak için pratiktir.

```

1 #include <stdio.h>
2 int main(void) {
3     // "r": komutun STDOUT'unu oku. ("w": komutun STDIN'ine yaz)
4     FILE *pipe_fp = popen("ls -l /etc | head -5", "r");
5     if (!pipe_fp) { perror("popen"); return 1; }
6
7     char line[256];
8     while (fgets(line, sizeof line, pipe_fp)) // ciktiyi satir satir oku
9         printf("--> %s", line);
10
11    int status = pclose(pipe_fp); // kapat + cikis kodunu al
12    printf("Komut cikis kodu: %d\n", WEXITSTATUS(status));
13    return 0;
14 }
```

21.3 Elle Çıktı Yakalama: fork + exec + pipe

popen kabuk kullanır; tam kontrol ve güvenlik için boruyu kendin kurarsın. Bu, bir komutu çalıştırıp çıktısını tampona toplamanın "endüstriyel" yoludur.

```

1 #include <unistd.h>
2 #include <sys/wait.h>
3 #include <stdio.h>
4 #include <string.h>
```

```

5
6 int main(void) {
7     int pipe_fd[2];
8     if (pipe(pipe_fd) == -1) { perror("pipe"); return 1; }
9
10    pid_t pid = fork();
11    if (pid == 0) {
12        // COCUK: komutu calistir
13        close(pipe_fd[0]); // okuma ucu gereksiz
14        dup2(pipe_fd[1], STDOUT_FILENO); // stdout -> borunun yazma ucu
15        close(pipe_fd[1]);
16        char *av[] = {"date", "+%Y-%m-%d %H:%M:%S", NULL};
17        execvp("date", av); // kabuk YOK: guvenli
18        _exit(127); // exec basarisizsa
19    }
20
21    // EBEVEYN: cocugun ciktisini oku
22    close(pipe_fd[1]); // yazma ucu gereksiz
23    char output[512];
24    ssize_t total = 0, n;
25    while ((n = read(pipe_fd[0], output + total, sizeof output - 1 - total)) > 0)
26        total += n;
27    output[total] = '\0';
28    close(pipe_fd[0]);
29
30    int status;
31    waitpid(pid, &status, 0); // zombi birakma
32    printf("Yakalanan cikti: %s", output);
33    printf("Cikis kodu: %d\n", WEXITSTATUS(status));
34    return 0;
35 }

```

Bilgi

Bu desen system/popen'a göre daha uzundur ama: (1) kabuk enjeksiyonu riski yoktur, (2) çıkış kodunu ve stderr'i ayrı yakalayabilirsin, (3) zaman aşımı/sinyal kontrolü sende olur. Üretim kodunda tercih edilen yol budur.

21.4 Basit Bir Kabuk (Mini-Shell)

Öğrenilen her şeyi birleştiren klasik alıştırma: komut oku, ayrıştır, fork/exec ile çalıştır, bekle — döngü.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <sys/wait.h>
6
7 int main(void) {
8     char line[1024];
9     while (1) {
10        printf("minish$ ");
11        fflush(stdout);
12        if (!fgets(line, sizeof line, stdin)) break; // Ctrl+D -> cik
13        line[strcspn(line, "\n")] = '\0'; // sondaki \n'i at
14        if (line[0] == '\0') continue;
15        if (strcmp(line, "exit") == 0) break; // dahili komut
16    }

```

```

17 // Bosluklara gore argumanlara ayir
18 char *argv[64]; int argc = 0;
19 char *tok = strtok(line, " ");
20 while (tok && argc < 63) { argv[argc++] = tok; tok = strtok(NULL, " "); }
21 argv[argc] = NULL;
22
23 pid_t pid = fork();
24 if (pid == 0) { // COCUK
25     execvp(argv[0], argv);
26     perror(argv[0]); // komut bulunamadi
27     _exit(127);
28 } else if (pid > 0) { // EBEVEYN
29     int status;
30     waitpid(pid, &status, 0); // komutun bitmesini bekle
31 }
32 }
33 printf("\nHosca kal.\n");
34 return 0;
35 }

```

İpucu

Bu mini-shell'i geliştirmek harika bir egzersizdir: boru (|), yönlendirme (>, <, 2>), arka plan çalıştırma (&), cd gibi dahili komutlar ve sinyal yönetimi ekleyerek gerçek bir kabuğa dönüştürebilirsin.

22. Sinyaller

Sinyal, bir sürece gönderilen asenkron bildirimdir: Ctrl+C (SIGINT), zamanlayıcı (SIGALRM), seg-fault (SIGSEGV), çocuk bitişi (SIGCHLD) gibi.

22.1 Yaygın Sinyaller

Sinyal	No	Anlam / Varsayılan
SIGINT	2	Ctrl+C; sonlandır.
SIGKILL	9	Zorla öldür; yakalanamaz/engellenemez.
SIGSEGV	11	Geçersiz bellek erişimi; core dump.
SIGTERM	15	Kibar sonlandırma isteği (varsayılan kill).
SIGCHLD	17	Çocuk süreç durdu/bitti.
SIGALRM	14	alarm() zamanlayıcısı doldu.
SIGSTOP / SIGCONT	19/18	Durdur / devam ettir.

22.2 sigaction ile Sinyal Yakalama

signal() taşınabilirlik açısından güvenilmezdir; modern kod sigaction kullanır.

```

1 #include <stdio.h>
2 #include <signal.h>
3 #include <unistd.h>

```

```

4
5 volatile sig_atomic_t running = 1;      // handler ile paylasilan bayrak
6
7 void handler(int sig) {
8     // Handler icinde yalnızca async-signal-safe islemler yapilmali.
9     // Bir bayrak set etmek guvenlidir; printf DEGILDIR.
10    running = 0;
11    (void)sig;
12 }
13
14 int main(void) {
15     struct sigaction sa = {0};
16     sa.sa_handler = handler;
17     sigemptyset(&sa.sa_mask);          // handler sirasinda bloklanacak sinyaller
18     sa.sa_flags = SA_RESTART;          // kesilen sistem cagrilarini otomatik tekrarla
19     sigaction(SIGINT, &sa, NULL);      // Ctrl+C'yi yakala
20
21     printf("Calisiyor... Ctrl+C ile durdur.\n");
22     while (running) {
23         pause();                        // bir sinyal gelene kadar uyu
24     }
25     printf("\nDuzgun sekilde cikildi.\n");
26     return 0;
27 }

```

Dikkat

Sinyal handler'ı içinde *yalnızca* async-signal-safe fonksiyonlar çağrılabilir (write evet, printf/malloc **hayır**). Handler ile ana kod arasında paylaşılan değişken mutlaka volatile sig_atomic_t olmalıdır.

22.3 kill ve alarm

```

1 #include <signal.h>
2 #include <unistd.h>
3 #include <stdio.h>
4
5 void on_timeout(int s) { (void)s; write(2, "Zaman doldu!\n", 13); }
6
7 int main(void) {
8     struct sigaction sa = {0};
9     sa.sa_handler = on_timeout;
10    sigaction(SIGALRM, &sa, NULL);
11
12    alarm(3);          // 3 saniye sonra SIGALRM gonder
13    printf("Bir sey bekleniyor (en fazla 3 sn)...\n");
14    pause();           // sinyali bekle
15
16    // Baska bir surece sinyal gondermek:
17    // kill(hedef_pid, SIGTERM);
18    return 0;
19 }

```

22.4 SIGCHLD ile Zombi Önleme

```

1 #include <signal.h>

```

```

2 #include <sys/wait.h>
3
4 void on_child_exit(int s) {
5     (void)s;
6     // Biriken tum bitmis cocuklari topla (WNOHANG: bloklanma)
7     while (waitpid(-1, NULL, WNOHANG) > 0)
8         ;
9 }
10 // main icinde: sigaction ile SIGCHLD -> on_child_exit kurulur.
11 // Boylece cocuklar otomatik toplanir, zombi kalmaz.

```

23. Borular (Pipes) ve FIFO

Boru, tek yönlü bir bayt akışıdır: bir uçtan yazılır, diğerinden okunur. İlişkili süreçler (fork ile) arası iletişimin en basit yoludur.

23.1 Anonim Boru (pipe)

```

1 #include <unistd.h>
2 #include <stdio.h>
3 #include <string.h>
4 int main(void) {
5     int fd[2];           // fd[0]=okuma ucu, fd[1]=yazma ucu
6     if (pipe(fd) == -1) { perror("pipe"); return 1; }
7
8     pid_t pid = fork();
9     if (pid == 0) {      // COCUK: yazar
10         close(fd[0]);    // okuma ucunu kapat
11         const char *message = "Merhaba ebeveyn!";
12         write(fd[1], message, strlen(message));
13         close(fd[1]);
14     } else {             // EBEVEYN: okur
15         close(fd[1]);    // yazma ucunu kapat
16         char buffer[128];
17         ssize_t n = read(fd[0], buffer, sizeof(buffer) - 1);
18         if (n > 0) { buffer[n] = '\0'; printf("Alindi: %s\n", buffer); }
19         close(fd[0]);
20         wait(NULL);
21     }
22     return 0;
23 }

```

Dikkat

Kullanmadığın boru uçlarını *mutlaka kapat*. Yazma ucu tümüyle kapanmazsa okuyan taraf EOF (0) alamaz ve sonsuza kadar bekler. Bu, en sık yapılan boru hatasıdır.

23.2 Boru + exec: Kabuğun "|" Operatörü

dup2 ile bir sürecin stdout'unu diğerinin stdin'ine bağlayarak `ls | wc -l` eşdeğerini kurabiliriz.

```

1 #include <unistd.h>
2 #include <sys/wait.h>
3 int main(void) {
4     int fd[2];

```



```

5   pipe(fd);
6
7   if (fork() == 0) {           // 1. çocuk: "ls"
8       dup2(fd[1], STDOUT_FILENO); // stdout -> borunun yazma ucu
9       close(fd[0]); close(fd[1]);
10      execlp("ls", "ls", NULL);
11      _exit(127);
12  }
13  if (fork() == 0) {           // 2. çocuk: "wc -l"
14      dup2(fd[0], STDIN_FILENO); // stdin -> borunun okuma ucu
15      close(fd[0]); close(fd[1]);
16      execlp("wc", "wc", "-l", NULL);
17      _exit(127);
18  }
19  close(fd[0]); close(fd[1]); // ebeveyn iki ucu da kapatır
20  wait(NULL); wait(NULL);    // iki cocugu bekle
21  return 0;
22 }

```

23.3 Adlandırılmış Boru (FIFO)

FIFO, dosya sisteminde bir isme sahip boru olduğu için *ilişkisiz* süreçler de kullanabilir.

```

1  // --- yazar.c ---
2  #include <sys/stat.h>
3  #include <fcntl.h>
4  #include <unistd.h>
5  int main(void) {
6      mkfifo("/tmp/borum", 0666); // FIFO olustur (bir kez yeter)
7      int fd = open("/tmp/borum", O_WRONLY); // okuyucu baglanana kadar bloke
8      write(fd, "FIFO uzerinden selam", 20);
9      close(fd);
10     return 0;
11 }
12 // --- okuyan.c ---
13 // int fd = open("/tmp/borum", O_RDONLY);
14 // char buf[64]; ssize_t n = read(fd, buf, sizeof buf); ...

```

23.4 Çift Yönlü İletişim (İki Boru)

Bir boru tek yönlüdür. İki süreç arasında karşılıklı konuşma için *iki* boru kurulur — biri her yön için.

```

1  #include <unistd.h>
2  #include <sys/wait.h>
3  #include <stdio.h>
4  #include <string.h>
5
6  int main(void) {
7      int p2c[2], c2p[2]; // ebeveyn->cocuk, cocuk->ebeveyn
8      pipe(p2c); pipe(c2p);
9
10     if (fork() == 0) { // COCUK
11         close(p2c[1]); close(c2p[0]); // kullanılmayan uclari kapat
12         char buf[64];
13         ssize_t n = read(p2c[0], buf, sizeof buf - 1);
14         buf[n] = '\0';
15         printf("Cocuk aldi: %s\n", buf);

```

```

16     write(c2p[1], "selam ebeveyn", 13); // cevap yaz
17     _exit(0);
18 } else { // EBEVEYN
19     close(p2c[0]); close(c2p[1]);
20     write(p2c[1], "selam cocuk", 11); // once yaz
21     char buf[64];
22     ssize_t n = read(c2p[0], buf, sizeof buf - 1);
23     buf[n] = '\0';
24     printf("Ebeveyn aldi: %s\n", buf);
25     wait(NULL);
26 }
27 return 0;
28 }

```

Dikkat

Çift yönlü boruda **kilitlenme** (deadlock) riski vardır: iki taraf da aynı anda okuma beklerse ya da tampon dolarken ikisi de yazmaya çalışırsa program donar. Protokolü net tanımla (kim önce yazar/okur) ve gerekirse non-blocking G/Ç kullan.

23.5 N Komutluk Boru Hattı

Kabuğun `a | b | c` zincirini kurmak için $N-1$ boru gerekir; her komut öncekinin çıktısından okuyup sonrakine yazar.

```

1  #include <unistd.h>
2  #include <sys/wait.h>
3  #include <stdio.h>
4
5  // Ornek: cat /etc/passwd | grep root | wc -l
6  int main(void) {
7      char *commands[][4] = {
8          {"cat", "/etc/passwd", NULL},
9          {"grep", "root", NULL},
10         {"wc", "-l", NULL},
11     };
12     int count = 3;
13     int prev_read = -1; // ilk komutun girisi normal stdin
14
15     for (int i = 0; i < count; i++) {
16         int pipe_fd[2];
17         int last = (i == count - 1);
18         if (!last) pipe(pipe_fd); // son komut disinda boru ac
19
20         if (fork() == 0) { // COCUK
21             if (prev_read != -1) { // ortadaki/son: onceki borudan oku
22                 dup2(prev_read, STDIN_FILENO);
23                 close(prev_read);
24             }
25             if (!last) { // son degilse: bir sonraki boruya yaz
26                 close(pipe_fd[0]);
27                 dup2(pipe_fd[1], STDOUT_FILENO);
28                 close(pipe_fd[1]);
29             }
30             execvp(commands[i][0], commands[i]);
31             _exit(127);
32         }
33         // EBEVEYN: kullanilmayan uclari kapat, sonraki iterasyona hazirla

```

```

34     if (prev_read != -1) close(prev_read);
35     if (!last) { close(pipe_fd[1]); prev_read = pipe_fd[0]; }
36 }
37 for (int i = 0; i < count; i++) wait(NULL); // tum cocuklari bekle
38 return 0;
39 }

```

23.6 SIGPIPE: Kapalı Boruya Yazma

Okuma ucu kapalı bir boruya yazmak SIGPIPE sinyali üretir; varsayılan davranış programı *öldürmek*-tir. Ağ ve boru kodunda bu sinyal yönetilmelidir.

```

1  #include <signal.h>
2  #include <unistd.h>
3  #include <errno.h>
4  #include <stdio.h>
5  int main(void) {
6      signal(SIGPIPE, SIG_IGN); // sinyali yok say -> write EPIPE dondursun
7
8      int pipe_fd[2];
9      pipe(pipe_fd);
10     close(pipe_fd[0]); // okuma ucunu kapat
11
12     ssize_t r = write(pipe_fd[1], "veri", 4); // kimse okumuyor
13     if (r == -1 && errno == EPIPE)
14         printf("Yazma basarisiz: karsi uc kapali (EPIPE)\n");
15     close(pipe_fd[1]);
16     return 0;
17 }

```

PIPE_BUF ve Atomiklik

PIPE_BUF baytıdan (Linux'ta 4096) küçük yazmalar *atomiktir*: birden çok yazıcı olsa bile veriler iç içe geçmez. Daha büyük yazmalar bölünebilir. Çok yazıcılı boru kayıtlarını bu sınırın altında tut.

23.7 popen ile Boru — Hızlı Alternatif

Süreçler arası boruyu elle kurmak yerine, tek bir kabuk komutu için popen çok daha kısadır (Terminal Komutları bölümüne bakınız). Elle pipe+dup2 ise kabuk kullanmadan tam kontrol sağlar — güvenlik ve esneklik gerektiğinde onu seç.

24. IPC: Paylaşımlı Bellek, Mesaj Kuyruğu, Semafor

Süreçler arası iletişim (Inter-Process Communication) için borular dışında üç güçlü mekanizma vardır. Modern kod POSIX API'lerini tercih eder.

24.1 POSIX Paylaşımlı Bellek (shm_open + mmap)

En hızlı IPC'dir: iki süreç aynı fiziksel belleği görür; kopyalama yoktur.

```

1  #include <sys/mman.h>
2  #include <sys/stat.h>
3  #include <fcntl.h>
4  #include <unistd.h>
5  #include <stdio.h>

```

```

6  #include <string.h>
7
8  #define NAME "/paylasim"
9  #define SIZE 4096
10
11 int main(void) {
12     // Paylasimli bellek nesnesi olustur/ac
13     int fd = shm_open(NAME, O_CREAT | O_RDWR, 0666);
14     ftruncate(fd, SIZE); // boyutlandir
15
16     // Belegi surecin adres alanina esle
17     char *mem = mmap(NULL, SIZE, PROT_READ | PROT_WRITE,
18                       MAP_SHARED, fd, 0);
19     if (mem == MAP_FAILED) { perror("mmap"); return 1; }
20
21     if (fork() == 0) { // COCUK: yazar
22         strcpy(mem, "Paylasimli bellekten selam");
23         _exit(0);
24     } else { // EBEVEYN: okur
25         wait(NULL);
26         printf("Okundu: %s\n", mem);
27     }
28
29     munmap(mem, SIZE);
30     close(fd);
31     shm_unlink(NAME); // isimlendirilmis nesneyi sil
32     return 0;
33 }
34 // Derleme: gcc bu.c -o bu -lrt

```

Dikkat

Paylaşımlı bellekte *senkronizasyon yoktur*: iki süreç aynı anda yazarsa yarış durumu (race condition) oluşur. Erişimi bir semafor veya mutex ile korumalısın.

24.2 POSIX Semafor

Semafor, paylaşılan bir kaynağa erişimi sayan bir kilittir. `sem_wait` (azalt/bekle) ve `sem_post` (artır/serbest bırak) atomiktir.

```

1  #include <semaphore.h>
2  #include <fcntl.h>
3  #include <stdio.h>
4  #include <unistd.h>
5  #include <sys/wait.h>
6
7  int main(void) {
8     // Baslangic degeri 1 -> ikili (mutex gibi) semafor
9     sem_t *sem = sem_open("/semaforum", O_CREAT, 0666, 1);
10
11     if (fork() == 0) {
12         sem_wait(sem); // kritik bolgeye gir (kilitler)
13         printf("Cocuk kritik bolgede\n");
14         sleep(1);
15         sem_post(sem); // cik (birak)
16         _exit(0);
17     } else {
18         sem_wait(sem);

```

```

19     printf("Ebeveyn kritik bolgede\n");
20     sem_post(sem);
21     wait(NULL);
22 }
23 sem_close(sem);
24 sem_unlink("/semaforum");
25 return 0;
26 }
27 // Derleme: gcc bu.c -o bu -lpthread -lrt

```

24.3 POSIX Mesaj Kuyruğu

Mesaj kuyruğu, süreçler arasında *sınır korumalı* (mesaj mesaj) ve öncelikli veri aktarımı sağlar.

```

1 #include <queue.h>
2 #include <stdio.h>
3 #include <string.h>
4 int main(void) {
5     struct mq_attr attr = {0};
6     attr.mq_maxmsg = 10;           // kuyrukta en fazla 10 mesaj
7     attr.mq_msgsize = 256;         // her mesaj en fazla 256 bayt
8
9     mqd_t mq = mq_open("/kuyruk", O_CREAT | O_RDWR, 0666, &attr);
10
11     mq_send(mq, "birinci", 7, 1);   // (kuyruk, veri, uzunluk, oncelik)
12     mq_send(mq, "onemli", 6, 9);   // yuksek oncelik once alinir
13
14     char buf[256]; unsigned priority;
15     ssize_t n = mq_receive(mq, buf, sizeof buf, &priority);
16     printf("Alindi (%u): %.*s\n", priority, (int)n, buf); // onemli
17
18     mq_close(mq);
19     mq_unlink("/kuyruk");
20     return 0;
21 }
22 // Derleme: gcc bu.c -o bu -lrt

```

Mekanizma	Güçlü Yanı	Not
Boru/FIFO	Basit, akış temelli	Tek yönlü, mesaj sınırı yok
Paylaşımlı bellek	En hızlı	Senkronizasyon gerekir
Mesaj kuyruğu	Mesaj sınırı + öncelik	Kopyalama maliyeti
Semafor	Senkronizasyon	Veri taşımaz, koordine eder

25. İş Parçacıkları (POSIX Threads)

İş parçacığı (thread), aynı süreç içinde paralel çalışan bir yürütme akışıdır. Süreçlerin aksine iş parçacıkları *aynı bellek alanını* paylaşır — bu hem güçlü hem tehlikelidir.

25.1 İş Parçacığı Oluşturma

```

1 #include <pthread.h>
2 #include <stdio.h>
3
4 void *worker(void *arg) {
5     int id = *(int *)arg;
6     printf("İsci %d calisiyor\n", id);
7     return NULL;           // pthread_exit(NULL) ile ayni
8 }
9
10 int main(void) {
11     pthread_t threads[3];
12     int ids[3] = {0, 1, 2};
13
14     for (int i = 0; i < 3; i++)
15         pthread_create(&threads[i], NULL, worker, &ids[i]);
16
17     for (int i = 0; i < 3; i++)
18         pthread_join(threads[i], NULL);    // her ipligin bitmesini bekle
19
20     printf("Tum iplikler bitti\n");
21     return 0;
22 }
23 // Derleme: gcc bu.c -o bu -lpthread

```

25.2 Yarış Durumu (Race Condition)

Paylaşılan bir değişkene senkronizasyonsuz erişim, öngörülemez sonuç verir.

```

1 #include <pthread.h>
2 #include <stdio.h>
3 long counter = 0;           // paylasilan; KORUNMASIZ
4
5 void *increment(void *arg) {
6     (void)arg;
7     for (int i = 0; i < 1000000; i++)
8         counter++;          // counter++ atomik DEGIL: oku-artir-yaz
9     return NULL;
10 }
11 int main(void) {
12     pthread_t a, b;
13     pthread_create(&a, NULL, increment, NULL);
14     pthread_create(&b, NULL, increment, NULL);
15     pthread_join(a, NULL); pthread_join(b, NULL);
16     printf("sayac = %ld (beklenen 2000000)\n", counter); // genelde daha az!
17     return 0;
18 }

```

25.3 Mutex ile Koruma

```

1 #include <pthread.h>
2 #include <stdio.h>
3 long counter = 0;
4 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
5
6 void *increment(void *arg) {
7     (void)arg;
8     for (int i = 0; i < 1000000; i++) {

```

```

9     pthread_mutex_lock(&lock);          // kritik bolgeyi kilitle
10    counter++;
11    pthread_mutex_unlock(&lock);        // birak
12  }
13  return NULL;
14 }
15 int main(void) {
16     pthread_t a, b;
17     pthread_create(&a, NULL, increment, NULL);
18     pthread_create(&b, NULL, increment, NULL);
19     pthread_join(a, NULL); pthread_join(b, NULL);
20     printf("sayac = %ld\n", counter);    // her zaman 2000000
21     pthread_mutex_destroy(&lock);
22     return 0;
23 }

```

Dikkat

Deadlock (kilitlenme): iki iş parçacığı birbirinin tuttuğu kilidi beklerse ikisi de sonsuza kadar durur. Önlem: tüm kilitleri *her yerde aynı sırada* al ve kritik bölgeyi kısa tut.

25.4 Koşul Değişkeni (Condition Variable)

Üretici-tüketici gibi senaryolarda, bir iş parçacığının bir koşulu beklemesi için `pthread_cond_t` kullanılır.

```

1  #include <pthread.h>
2  #include <stdio.h>
3
4  pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
5  pthread_cond_t c = PTHREAD_COND_INITIALIZER;
6  int ready = 0;
7
8  void *consumer(void *arg) {
9      (void)arg;
10     pthread_mutex_lock(&m);
11     while (!ready)                // DAIMA dongude bekle (spurious wakeup)
12         pthread_cond_wait(&c, &m); // kilidi birakip uyur, uyanınca kilitler
13     printf("Tuketici: veri hazır!\n");
14     pthread_mutex_unlock(&m);
15     return NULL;
16 }
17 void *producer(void *arg) {
18     (void)arg;
19     pthread_mutex_lock(&m);
20     ready = 1;
21     pthread_cond_signal(&c);      // bekleyeni uyandır
22     pthread_mutex_unlock(&m);
23     return NULL;
24 }
25 int main(void) {
26     pthread_t t, u;
27     pthread_create(&t, NULL, consumer, NULL);
28     pthread_create(&u, NULL, producer, NULL);
29     pthread_join(t, NULL); pthread_join(u, NULL);
30     return 0;
31 }

```

İpucu

`pthread_cond_wait` her zaman bir `while` döngüsü içinde kullanılır çünkü "sahte uyanma" (spurious wakeup) mümkündür. `if` ile kontrol *hatalıdır*.

25.5 İş Parçacığı mı, Süreç mi?

İş Parçacığı (thread)	Süreç (process)
Belleği paylaşır (hızlı iletişim)	Ayrı bellek (izolasyon, güvenlik)
Oluşturması ucuz	Oluşturması pahalı (fork)
Biri çökerse tümü çöker	Biri çökerse diğerleri sağ kalır
Senkronizasyon şart	Doğal olarak yalıtılmış

26. Atomik İşlemler (Atomics)

Önceki bölümde `sayac++` işleminin iş parçacıkları arasında *yarış durumu* ürettiğini gördük. Nedeni şu: `sayac++` tek bir işlem gibi görünse de derleyici bunu üç ayrı makine adımına çevirir — **oku**, **artır**, **yaz**. İki iş parçacığı bu adımları iç içe çalıştırırsa güncellemelerden biri kaybolur.

```

1 // counter = 41 iken iki iplik aynı anda counter++ yaparsa:
2 // Iplik A: oku(41)           Iplik B: oku(41)
3 // Iplik A: artir -> 42       Iplik B: artir -> 42
4 // Iplik A: yaz(42)           Iplik B: yaz(42) // iki artis, sonuc 42!

```

Bunu mutex ile çözebiliriz ama mutex pahalıdır: kilit alıp bırakmak sistem çağrısına dönüşebilir, iş parçacıkları uyutulup uyandırılır. **Atomik işlemler**, tek bir değişken üzerindeki oku-değiştir-yaz adımlarını donanımın *bölünemez* tek bir talimatıyla (ör. x86 `lock xadd`) gerçekleştirir. Kilit yoktur, uyuma yoktur — çok daha hızlıdır.

Bilgi

Atomik bir işlem **ya tamamen olur ya hiç olmaz**; ortada başka bir iş parçacığı onu yarım halde göremez. C11 ile dile giren `<stdatomic.h>` başlığı bunu taşınabilir biçimde sunar.

26.1 _Atomic ve <stdatomic.h>

Bir değişkeni `_Atomic` niteleyicisiyle tanımlarsanız, üzerindeki sıradan işlemler (`++`, `+=`, `atama`) otomatik olarak atomik olur. Yarış durumu örneğini tek kelime değiştirerek düzeltelim:

```

1 #include <pthread.h>
2 #include <stdatomic.h>
3 #include <stdio.h>
4
5 _Atomic long counter = 0;           // artık ++ atomik: yarış yok
6
7 void *increment(void *arg) {
8     (void)arg;
9     for (int i = 0; i < 1000000; i++)
10         counter++;                 // lock xadd -> bölünemez oku-artir-yaz
11     return NULL;

```



```

12 }
13 int main(void) {
14     pthread_t a, b;
15     pthread_create(&a, NULL, increment, NULL);
16     pthread_create(&b, NULL, increment, NULL);
17     pthread_join(a, NULL); pthread_join(b, NULL);
18     printf("sayac = %ld (beklenen 2000000)\n", counter); // HER ZAMAN 2000000
19     return 0;
20 }
21 // Derleme: gcc bu.c -o bu -lpthread (C11 gerekli, gcc varsayılan destekler)

```

İpucu

atomic_int, atomic_long, atomic_bool, atomic_size_t gibi hazır tür adları <stdatomic.h> içinde tanımlıdır; _Atomic int ile aynı anlama gelir.

26.2 Açık Atomik Fonksiyonlar

++ operatörü okunaklıdır ama dönüş değeriyle çalışmak veya niyeti açıkça göstermek için fonksiyon biçimleri vardır. Hepsi *eski değeri* döndürür:

```

1 #include <stdatomic.h>
2
3 atomic_int x = 0;
4
5 atomic_store(&x, 10); // x = 10
6 int e = atomic_load(&x); // e = 10 (guvenli okuma)
7
8 int old = atomic_fetch_add(&x, 5); // x -> 15, eski = 10
9 atomic_fetch_sub(&x, 3); // x -> 12
10 atomic_fetch_or(&x, 0x1); // bit maskeleme de atomik
11 atomic_fetch_and(&x, ~0x2);
12
13 int y = atomic_exchange(&x, 99); // x'e 99 yaz, eski degeri dondur

```

Not

Neden dönüş değeri önemli? Çok iş parçacıklı bir sayacıta "benim artırdığım andaki değer neydi?" sorusuna sadece fetch_add'ın dönüşü doğru yanıtı verir. Önce load sonra store yaparsanız araya başka iplik girebilir — yine yarış.

26.3 Compare-and-Swap (CAS): Kilitlessiz Programlamanın Temeli

En güçlü atomik ilkel **karşılaştır-ve-değiştirir**: atomic_compare_exchange_weak(&nesne, &beklenen, yeni). Anlamı: "nesne hâlâ beklenen'e eşitse yeni'yi yaz ve true döndür; değilse beklenen'i nesnenin *güncel* değeriyle güncelle ve false döndür." Bu, "değiştirmeden önce kimse dokunmadığından emin ol" mantığıdır ve kilitlessiz veri yapılarının kalbidir.

Klasik desen bir CAS döngüsüdür — başarıya kadar tekrar dene:

```

1 #include <stdatomic.h>
2
3 // Atomik olarak toplama uygula (ornek amaclı; gercekte fetch_add kullanın)
4 void atomic_add(atomic_int *target, int value) {
5     int expected = atomic_load(target);
6     int desired;
7     do {
8         desired = expected + value;

```

```

9      // hedef hala 'beklenen' ise 'yeni'yi yaz; değilse beklenen guncellenir
10     } while (!atomic_compare_exchange_weak(target, &expected, desired));
11 }

```

Dikkat

`compare_exchange_weak` nadiren *sahte başarısızlık* verebilir (değer eşit olsa bile `false` döner); bu yüzden **daima bir döngü içinde** kullanılır. Döngüsüz tek atış gerekiyorsa `compare_exchange_strong` tercih edin.

Uygulama: Kilitsiz (lock-free) yığın

CAS ile mutex kullanmadan iş parçacığı-güvenli bir yığın (stack) yazılabilir. `push`, yeni düğümün `next`'ini mevcut tepeye bağlar, sonra tepeyi CAS ile günceller — araya biri girdiyse döngü tekrar dener:

```

1 #include <stdatomic.h>
2 #include <stdlib.h>
3
4 typedef struct Node { int data; struct Node *next; } Node;
5 _Atomic(Node *) top = NULL;           // atomik işaretçi
6
7 void push(int v) {
8     Node *new_node = malloc(sizeof(Node));
9     new_node->data = v;
10    new_node->next = atomic_load(&top);
11    // tepe hala yeni->next ise yeni'yi tepe yap; değilse yeni->next tazelenir
12    while (!atomic_compare_exchange_weak(&top, &new_node->next, new_node))
13        ;                               // bos govde: beklemeden yeniden dene
14 }
15
16 int pop(int *out) {                    // 1: basarili, 0: bos
17     Node *old = atomic_load(&top);
18     while (old && !atomic_compare_exchange_weak(&top, &old, old->next))
19         ;                               // eski, guncel tepeyle tazelenir
20     if (!old) return 0;
21     *out = old->data;
22     free(old);                          // NOT: ABA problemine dikkat (asagi)
23     return 1;
24 }

```

Dikkat

ABA problemi: Bir iplik tepeyi A okur, uyur; başka iplik A'yı çıkarıp serbest bırakır, sonra aynı adreste yeni bir A düğümü oluşur. İlk iplik uyanınca CAS "hâlâ A" diye başarılı olur ama artık *farklı* bir A'dır — liste bozulur. Gerçek kilitsiz yapılar bunu etiketli işaretçi (tagged pointer) veya *hazard pointer* / RCU ile çözer. Kilitsiz programlama uzmanlık ister; çoğu durumda mutex daha güvenli ve yeterlidir.

26.4 Bellek Sıralaması (Memory Ordering)

Modern CPU'lar ve derleyiciler, hızı artırmak için bellek işlemlerinin *sırasını* değiştirebilir. Tek iş parçacığında fark edilmez, ama çok iş parçacığında bir ipliğin yazdıklarını başka ipliğin *hangi sırayla* gördüğü önemlidir. Her atomik fonksiyon, ikinci argüman olarak bir `memory_order` alabilir:

Sıralama	Anlamı / Kullanımı
memory_order_relaxed	Yalnızca atomiklik; sıralama garantisi yok. Sadece sayaç (ör. istatistik) için idealdir.
memory_order_acquire	Bir <i>okuma</i> bariyeri: bu yükten sonraki erişimler öne kaymaz. Kilit <i>alırken</i> .
memory_order_release	Bir <i>yazma</i> bariyeri: bu depodan önceki erişimler geriye kaymaz. Kilit <i>bırakırken</i> .
memory_order_acq_rel	Hem acquire hem release; oku-değiştir-yaz için.
memory_order_seq_cst	En güçlü: tüm iplikler için tek bir küresel sıra. Varsayılan ve en güvenli.

sayac++ gibi kısayollar her zaman seq_cst kullanır — en güvenli ama en yavaş olanı. Sırf saymak yeterliyse relaxed çok daha hızlıdır:

```

1 #include <stdatomic.h>
2
3 atomic_long op_count = 0;
4
5 void on_event(void) {
6     // Sadece toplam önemli; hangi sırada arttığı onemsiz -> relaxed
7     atomic_fetch_add_explicit(&op_count, 1, memory_order_relaxed);
8 }

```

Klasik acquire/release deseni: bir iplik veriyi hazırlar, sonra bir bayrağı release ile set eder; diğer iplik bayrağı acquire ile okuduğunda verinin de hazır olduğunu görmesi **garantidir**:

```

1 #include <stdatomic.h>
2
3 int data = 0; // sıradan değişken
4 atomic_bool ready = false;
5
6 void producer(void) {
7     data = 42; // (1) önce veriyi yaz
8     atomic_store_explicit(&ready, true, memory_order_release); // (2) sonra bayrağı set et
9 }
10
11 void consumer(void) {
12     while (!atomic_load_explicit(&ready, memory_order_acquire))
13         ; // bayrağı bekle
14     // release/acquire eşleşmesi: (1) burada KESİN görünür
15     printf("veri = %d\n", data); // daima 42
16 }

```

Bilgi

Kural: emin değilseniz memory_order vermeyin — varsayılan seq_cst her zaman doğrudur, sadece biraz yavaştır. relaxed ve acquire/release ölçülmüş bir darboğaz varsa ve ne yaptığınızı biliyorsanız kullanılır.

26.5 atomic_flag ile Spinlock

atomic_flag, standardın *kilitsiz olması garanti edilen* tek türüdür. test_and_set bayrağı set eder ve *önceki* değerini döndürür; bununla basit bir spinlock (dönen kilit) yazılır:

```

1 #include <stdatomic.h>
2
3 atomic_flag lock = ATOMIC_FLAG_INIT;
4
5 void spin_lock(void) {
6     // set edip eski deger true ise baskasi tutuyor demektir -> don ve bekle
7     while (atomic_flag_test_and_set_explicit(&lock, memory_order_acquire))
8         ; // "busy-wait" (mesgul bekleme)
9 }
10 void spin_unlock(void) {
11     atomic_flag_clear_explicit(&lock, memory_order_release);
12 }

```

Dikkat

Spinlock CPU'yu döngüde *meşgul* tutar (uyumaz). Yalnızca kritik bölge çok kısaysa ve çekişme düşükse mutex'ten hızlıdır; uzun beklemelerde CPU'yu boşa yakar. Genel amaçlı senkronizasyon için `pthread_mutex_t` kullanın.

26.6 GCC/Clang Yerleşikleri (__atomic)

C11'den önce (ve hâlâ yaygın olarak) GCC/Clang, `__atomic_*` ve eski `__sync_*` yerleşik fonksiyonlarını sunardı. Linux çekirdeği ve birçok eski kod tabanı bunları kullanır:

```

1 long counter = 0; // _Atomic gerekmez
2
3 __atomic_fetch_add(&counter, 1, __ATOMIC_SEQ_CST); // C11 fetch_add karşılığı
4 long v = __atomic_load_n(&counter, __ATOMIC_ACQUIRE);
5 __atomic_store_n(&counter, 0, __ATOMIC_RELEASE);
6
7 // Eski __sync ailesi (her zaman tam bariyer):
8 __sync_fetch_and_add(&counter, 1);
9 __sync_bool_compare_and_swap(&counter, 0, 100); // CAS

```

İpucu

Yeni kod yazıyorsanız taşınabilir C11 `<stdatomic.h>`'i tercih edin. `__atomic_*` yerleşiklerini yalnızca derleyiciye özgü kodda veya `_Atomic` niteleyemeyeceğiniz mevcut bir değişkende kullanın.

26.7 volatile Atomik Değildir!

Sık yapılan bir hata: `volatile` sanki iş parçacığı güvenliği sağlarmış gibi kullanmak. `volatile` yalnızca "bu değişkeni önbelleğe alma, her seferinde bellekten oku" der (donanım yazmaçları, sinyal handler'ları için). Atomiklik veya bellek sıralaması *sağlamaz*.

Özellik	volatile	_Atomic
Oku-değiştir-yaz bölünemez mi?	Hayır	Evet
Bellek sıralaması	Yok	Var (memory_order)
Kullanım amacı	Donanım/sinyal ("her defa oku")	İş parçacığı senkronizasyonu
sig_atomic_t ile	Sinyal handler bayrağı	—

Not

Sinyal handler'ları için hâlâ volatile sig_atomic_t doğru seçimdir (tek yazar, atomik okuma/yazma yeterli). Ama iş parçacıkları arasında paylaşım için _Atomic veya mutex kullanın.

26.8 Ne Zaman Hangisi?

Durum	Öneri
Tek bir sayaç / bayrak	_Atomic (en hızlı, en basit)
Birden çok değişkeni birlikte tutarlı güncellemek	pthread_mutex_t
Kritik bölge uzun / sistem çağrısı içeriyor	Mutex (spinlock değil)
Çok kısa kritik bölge, düşük çekişme	Spinlock (atomic_flag)
Kilitsiz veri yapısı	CAS döngüsü + ABA'ya dikkat (uzmanlık ister)

Bilgi

Altın kural: **en basit doğru araçtan başla.** Önce tek atomik, yetmezse mutex. Kilitsiz CAS koduna ancak profileme gerçek bir darboğaz gösterirse geç — doğrulaması zordur ve ABA gibi tuzaklar barındırır.

27. Ağ Programlama: Soketler

Soket, ağ üzerinden (ya da aynı makinede) iletişim uç noktasıdır. TCP güvenilir bağlantı akışı, UDP ise bağlantısız datagram sunar.

27.1 TCP Sunucu

```

1  #include <sys/socket.h>
2  #include <netinet/in.h>
3  #include <arpa/inet.h>
4  #include <unistd.h>
5  #include <string.h>
6  #include <stdio.h>
7
8  int main(void) {
9      // 1) Soket olustur (IPv4, TCP)
10     int s = socket(AF_INET, SOCK_STREAM, 0);
11     if (s == -1) { perror("socket"); return 1; }
12
13     // Adresi hemen tekrar kullanılabilir yap (TIME_WAIT sorununu asar)
14     int opt = 1;
15     setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof opt);
16
17     // 2) Adrese bagla (bind)
18     struct sockaddr_in addr = {0};
19     addr.sin_family = AF_INET;

```

```

20     addr.sin_addr.s_addr = INADDR_ANY;        // tum arayuzler
21     addr.sin_port = htons(8080);              // port (ag bayt sirasi!)
22     if (bind(s, (struct sockaddr *)&addr, sizeof addr) == -1) {
23         perror("bind"); return 1;
24     }
25
26     // 3) Dinlemeye basla (listen)
27     listen(s, 10);                          // 10 baglanti kuyruk boyu
28     printf("8080 portu dinleniyor...\n");
29
30     // 4) Baglanti kabul et (accept) -> yeni bir fd doner
31     struct sockaddr_in client;
32     socklen_t len = sizeof client;
33     int c = accept(s, (struct sockaddr *)&client, &len);
34
35     char ip[INET_ADDRSTRLEN];
36     inet_ntop(AF_INET, &client.sin_addr, ip, sizeof ip);
37     printf("Baglanti: %s:%d\n", ip, ntohs(client.sin_port));
38
39     // 5) Veri al/gonder
40     char buffer[1024];
41     ssize_t n = recv(c, buffer, sizeof buffer - 1, 0);
42     if (n > 0) { buffer[n] = '\0'; printf("Istemci: %s\n", buffer); }
43     const char *response = "Merhaba istemci!\n";
44     send(c, response, strlen(response), 0);
45
46     close(c);
47     close(s);
48     return 0;
49 }

```

27.2 TCP İstemci

```

1  #include <sys/socket.h>
2  #include <netinet/in.h>
3  #include <arpa/inet.h>
4  #include <unistd.h>
5  #include <string.h>
6  #include <stdio.h>
7
8  int main(void) {
9      int s = socket(AF_INET, SOCK_STREAM, 0);
10
11      struct sockaddr_in server = {0};
12      server.sin_family = AF_INET;
13      server.sin_port = htons(8080);
14      inet_pton(AF_INET, "127.0.0.1", &server.sin_addr); // metin -> ikili IP
15
16      if (connect(s, (struct sockaddr *)&server, sizeof server) == -1) {
17          perror("connect"); return 1;
18      }
19
20      const char *message = "Sunucuya selam";
21      send(s, message, strlen(message), 0);
22
23      char buffer[1024];
24      ssize_t n = recv(s, buffer, sizeof buffer - 1, 0);
25      if (n > 0) { buffer[n] = '\0'; printf("Sunucu: %s", buffer); }

```

```

26
27     close(s);
28     return 0;
29 }

```

Sunucu Adımı

İstemci Adımı

socket() → bind() → listen() → accept() socket() → connect()

Dikkat

Ağ üzerindeki tüm çok baytlı sayılar *ağ bayt sırasında* (big-endian) taşınır. Port ve adresleri htons/htonl ile gönder, ntohs/ntohl ile oku. Aksi halde farklı mimariler arası iletişim bozulur.

27.3 UDP: Bağlantısız İletişim

```

1 // --- UDP Sunucu (ozet) ---
2 int s = socket(AF_INET, SOCK_DGRAM, 0); // SOCK_DGRAM = UDP
3 // bind(...) aynı şekilde
4 struct sockaddr_in client; socklen_t len = sizeof client;
5 char buf[1024];
6 ssize_t n = recvfrom(s, buf, sizeof buf, 0,
7                     (struct sockaddr *)&client, &len); // kimden geldi?
8 sendto(s, "cevap", 5, 0, (struct sockaddr *)&client, len); // ona geri gönder
9 // UDP'de listen/accept/connect YOK; her datagram bagimsizdir.

```

28. mmap: Bellek Eşleme

mmap, bir dosyayı ya da anonim bir bellek bölgesini doğrudan sürecin adres alanına eşler. Dosyaya read/write yerine *diziymiş gibi* erişilir — büyük dosyalar için çok verimlidir.

```

1 #include <sys/mman.h>
2 #include <sys/stat.h>
3 #include <fcntl.h>
4 #include <unistd.h>
5 #include <stdio.h>
6
7 int main(int argc, char *argv[]) {
8     if (argc != 2) { fprintf(stderr, "Kullanim: %s dosya\n", argv[0]); return 1; }
9
10    int fd = open(argv[1], O_RDONLY);
11    if (fd == -1) { perror("open"); return 1; }
12
13    struct stat st;
14    fstat(fd, &st); // dosya boyutunu al
15
16    // Dosyayi salt-okunur olarak bellege esle
17    char *mapped = mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
18    if (mapped == MAP_FAILED) { perror("mmap"); close(fd); return 1; }
19    close(fd); // esleme sonrasi fd gereksiz
20

```

```

21 // Artık dosya bir char dizisi gibi: satirlari say
22 long lines = 0;
23 for (off_t i = 0; i < st.st_size; i++)
24     if (mapped[i] == '\n') lines++;
25 printf("Satir sayisi: %ld\n", lines);
26
27 munmap(mapped, st.st_size); // eslemeyi kaldır
28 return 0;
29 }

```

Bilgi

mmap ayrıca MAP_ANONYMOUS ile dosyasız bellek ayırmak (malloc'un altında kullanılır) ve MAP_SHARED ile süreçler arası paylaşımlı bellek için de kullanılır. PROT_WRITE ile eşlenirse belleğe yazmak dosyayı değiştirir.

29. Zaman ve Zamanlayıcılar

```

1 #include <stdio.h>
2 #include <time.h>
3 #include <unistd.h>
4
5 int main(void) {
6     time_t now = time(NULL); // 1970'ten beri saniye
7     printf("Unix zaman: %ld\n", (long)now);
8     printf("Yerel saat: %s", ctime(&now)); // insan-okur bicim
9
10    struct tm *t = localtime(&now); // parcalara ayir
11    printf("Yil: %d, Ay: %d, Gun: %d\n",
12           t->tm_year + 1900, t->tm_mon + 1, t->tm_mday);
13
14    // Yuksek cozunurluklu sure olcumu (monotonik saat)
15    struct timespec start, end;
16    clock_gettime(CLOCK_MONOTONIC, &start);
17    usleep(150000); // 150 ms uyu (mikrosaniye)
18    clock_gettime(CLOCK_MONOTONIC, &end);
19
20    double elapsed = (end.tv_sec - start.tv_sec)
21                   + (end.tv_nsec - start.tv_nsec) / 1e9;
22    printf("Gecen sure: %.3f sn\n", elapsed);
23    return 0;
24 }

```

İpucu

Süre ölçümünde CLOCK_MONOTONIC kullan; CLOCK_REALTIME (duvar saati) NTP ya da kullanıcı tarafından geri alınabilir ve negatif süreler ölçülmesine yol açabilir.

30. GDB ile Hata Ayıklama

GDB (GNU Debugger), çalışan bir programı durdurup adım adım yürütmeni, değişkenleri incelemeni, çökme yerini bulmanı ve belleği okumanı sağlar. Segfault, bellek bozulması ve mantık hatalarını

bulmanın en güçlü aracıdır.

30.1 Hazırlık: Doğru Derleme

```

1 # -g: hata ayıklama sembolleri (degisken adlari, satir no)
2 # -O0: optimizasyonu kapat (kod satirlarla ortusur, degisken kaybolmaz)
3 $ gcc -g -O0 -Wall program.c -o program
4
5 # Cekirdek (core) dump'lari etkinlestir (cokme sonrasi analiz icin)
6 $ ulimit -c unlimited
7
8 # GDB'yi baslat
9 $ gdb ./program          # programla baslat
10 $ gdb --args ./program a b # argumanlarla baslat
11 $ gdb -tui ./program      # TUI (metin arayuzu) ile baslat

```

30.2 Programı Çalıştırma ve Durdurma

Komut	Kısa	Görev
run [arg]	r	Programı baştan çalıştır.
start	—	main'de durup çalıştır (geçici kesme).
continue	c	Sonraki kesmeye kadar devam et.
kill	k	Çalışan programı durdur.
quit	q	GDB'den çık.
Ctrl+C	—	Çalışan programı anında durdur.

30.3 Kesme Noktaları (Breakpoints)

Komut	Görev
break main	main fonksiyonuna kesme koy (b main).
break dosya.c:42	42. satıra kesme koy.
break fonk if x>5	Koşullu kesme: yalnızca x>5 iken dur.
tbreak	Tek seferlik (geçici) kesme.
watch degisken	Watchpoint: değer her değiştiğinde dur.
rwatch / awatch	Okunduğunda / okunup-yazıldığında dur.
info breakpoints	Tüm kesmeleri listele (i b).
delete 2 / disable 2	2 no'lu kesmeyi sil / devre dışı bırak.

Bilgi

Watchpoint, bellek bozulması hatalarını bulmanın en güçlü yoludur: "bu değişken beklenmedik biçimde değişti" derken watch degisken koy, GDB değeri değiştiren *tam satırda* durur.

30.4 Adım Adım Yürütme (Stepping)

Komut	Kısa Görev
<code>next</code>	<code>n</code> Sonraki satır (fonksiyon çağrısının <i>üstünden</i> atla).
<code>step</code>	<code>s</code> Sonraki satır (fonksiyonun <i>içine</i> gir).
<code>finish</code>	<code>fin</code> Mevcut fonksiyondan çık (dönüş değerini göster).
<code>until 50</code>	<code>u 50</code> 50. satıra kadar çalıştır (döngüden çıkmak için).
<code>nexti / stepi</code>	<code>ni/si</code> Tek makine komutu ile (assembly seviyesi).
<code>return</code>	— Fonksiyondan hemen dön (geri kalanı atla).

Dikkat

`next` ile `step` farkı kritiktir: bir fonksiyon çağrısı satırında `next` onu tek adımda geçer, `step` ise içine dalar. Kütüphane fonksiyonlarına dalmamak için genelde `next` kullanılır.

30.5 Veriyi İnceleme (Print, Examine)

Komut	Görev
<code>print x</code>	<code>x</code> 'in değerini yazdır (<code>p x</code>).
<code>print/x x</code>	Onaltılık (hex) yazdır (<code>/d</code> ondalık, <code>/c</code> karakter, <code>/t</code> ikilik).
<code>print dizi@10</code>	Dizinin ilk 10 elemanını yazdır.
<code>print *ptr</code>	İşaretçinin gösterdiği değeri yazdır.
<code>print yapı</code>	Bir struct'ın tüm alanlarını yazdır.
<code>display x</code>	Her durakta <code>x</code> 'i otomatik göster.
<code>x/8xw &dizi</code>	Belleği incele: 8 word, hex (<code>x/</code> = examine).
<code>set var x = 5</code>	Değişkeni çalışırken değiştir.
<code>ptype x / whatis x</code>	Tipini göster.

`x/` (examine) biçimi

`x/NFU` adres — **N**=adet, **F**=biçim (`x`=hex, `d`=ondalık, `c`=karakter, `s`=string, `i`=komut), **U**=birim (`b`=bayt, `h`=2, `w`=4, `g`=8 bayt). Örnek: `x/16xb ptr` = 16 baytı hex olarak; `x/s ptr` = string olarak.

30.6 Çağrı Yığını (Backtrace)

Segfault olduğunda ilk yapılacak: `backtrace` ile "nereden nasıl buraya gelindi" zincirini görmek.

Komut	Kısa Görev
backtrace	bt Tüm çağrı yığını göster.
bt full	— Yığın + her çerçevedeki yerel değişkenler.
frame 2	f 2 2 no'lu çerçeveye geç.
up / down	— Bir üst / alt çerçeveye geç.
info locals	i lo Yereldeki tüm değişkenleri göster.
info args	i ar Fonksiyon argümanlarını göster.

Segfault'u bulma oturumu

Klasik akış: çalıştır, çök, backtrace, suçlu satırı ve değişkeni incele.

```

1 (gdb) run
2 Program received signal SIGSEGV, Segmentation fault.
3 0x0000... in list_add (head=0x0, data=5) at list.c:14
4 14          head->next = new_node;
5 (gdb) bt          # cagri yigini
6 #0 list_add (head=0x0, ...) at list.c:14
7 #1 0x... in main () at list.c:30
8 (gdb) print head          # suclu: bas NULL!
9 $1 = (Node *) 0x0
10 (gdb) frame 1          # cagirana cik
11 (gdb) print list          # neden NULL gecilmis?

```

30.7 TUI: Metin Kullanıcı Arayüzü ve Layout'lar

TUI modu, kaynağı/assembly'yi/register'ları GDB komut satırının üstünde bir pencerede gösterir — adım adım yürütmeyi görsel yapar.

Komut / Kısayol	Görev
Ctrl+X sonra A	TUI modunu aç/kapat.
layout src	Kaynak kodu penceresi.
layout asm	Assembly penceresi.
layout split	Kaynak + assembly birlikte.
layout regs	Register penceresi (+ kaynak).
Ctrl+X sonra 2	Pencere düzenini değiştir.
Ctrl+L	Ekranı yenile (bozulursa).
Ctrl+P / Ctrl+N	Komut geçmişinde geri/ileri.
focus cmd / focus src	Klavye odağını komut/kaynak penceresine ver.

İpucu

TUI'de yön tuşları odaklanan pencereyi kaydırır. Komut satırında geçmişe erişmek için focus cmd ile odağı komut penceresine al; kaynağı kaydırmak için focus src. Ekran bozulursa Ctrl+L yeniler.

30.8 Core Dump ve Çalışan Sürece Bağlanma

```

1 # 1) Cokme sonrasi (post-mortem) analiz: core dosyasiyla
2 $ ulimit -c unlimited           # once etkinlestir
3 $ ./program                     # cokup "core" uretir
4 $ gdb ./program core           # core'u yukle
5 (gdb) bt                       # cektugu andaki yigin
6
7 # 2) Zaten calisan bir surece baglanma (PID ile)
8 $ gdb -p 12345                  # 12345 PID'li surece baglan
9 (gdb) bt                       # o anki durumu incele
10 (gdb) detach                   # birak, surec calismaya devam etsin

```

30.9 Diğer Faydalı Komutlar

Komut	Görev
list / list 40	Kaynak kodu göster (1).
info registers	Tüm register değerleri (i r).
info functions / info variables	Sembolleri listele.
call fonk(3)	Program içindeki bir fonksiyonu elle çağır.
break + commands	Kesmeye ulaşınca otomatik komut dizisi çalıştır.
set print pretty on	Struct'ları girintili/okunur yazdır.
tbreak main; run	main'de dur (start ile aynı).

İş parçacığı hata ayıklama

Çok iş parçacıklı programlarda thread'ler arası geçiş.

```

1 (gdb) info threads             # tum iplikleri listele
2 (gdb) thread 3                 # 3 no'lu iplige gec
3 (gdb) thread apply all bt     # TUM ipliklerin yiginini goster (deadlock avi)

```

30.10 .gdbinit ile Otomasyon

Ev dizinindeki ~/.gdbinit dosyası her oturumda çalışır; sık kullanılan ayarları ve makroları kalıcı yapar.

```

1 # ~/.gdbinit ornegi
2 set print pretty on           # struct'lari okunur yazdir
3 set pagination off            # "--More--" duraklamasini kapat
4 set history save on           # komut gecmisini sakla
5
6 # Ozel komut (kisayol) tanimla:
7 define bt_full
8     thread apply all backtrace full
9 end

```

Bilgi

GDB alternatifleri: daha modern bir arayüz için **gdb -tui** ya da **cgdb**, Python tabanlı zengin görünüm için **gdb-dashboard** veya **pwndbg/GEF** eklentileri kullanılabilir. Bellek hataları için GDB'yi **AddressSanitizer** (`-fsanitize=address`) ile birlikte kullanmak, hatayı olduğu anda yakalar.

31. Bağlayıcı (Linker) Derinlemesine: ld, Kütüphaneler ve Linker Script

Bağlayıcı, nesne dosyalarını ve kütüphaneleri tek bir çalıştırılabilir (ya da paylaşımlı kütüphaneye) birleştiren araçtır. *Derleme Zamanı ve Çalışma Zamanı* bölümünde bağlamanın *ne zaman* olduğunu görmüştük; burada *nasıl* yönetildiğine iniyoruz: ld kullanımı, kendi kütüphaneni üretme ve belleği bayt bayt yerleştiren linker script'ler.

31.1 gcc ile ld ilişkisi: Kim Kimi Çağırır?

Pratikte ld'yi nadiren doğrudan çağırırsın; gcc onu senin için çalıştırır ve C çalışma zamanı nesnelerini (`crt1.o`, `crti.o`...) ile `libc` yollarını otomatik ekler. Elle ld çağırmak bunların hepsini kendin vermeni gerektirir — bu yüzden “sıradan” bağlamada gcc'yi sürücü olarak kullan, ld'ye özel seçenekleri `-Wl`, ile ilet.

```
1 $ gcc -v main.c -o main      # gcc'nin arka planda cagirdigi ld/collect2 satirini gorurusun
2 $ gcc main.o -o main         # onerilen: gcc surucu olarak (crt + libc otomatik)
3
4 # gcc'den ld'ye secenek gecirme: -Wl,<secenek>[,<arg>]
5 $ gcc main.o -o main -Wl,-Map=main.map      # ld'ye -Map=main.map ilet
6 $ gcc main.o -o main -Wl,--gc-sections      # kullanilmayan section'lari at
```

Dikkat

Bağlama sırası önemlidir: `-l` bayrağı, onu *kullanan* nesne dosyalarından *sonra* gelmelidir. `gcc -lm main.c` çoğu zaman “undefined reference to `sqrt`” verir; doğrusu `gcc main.c -lm`. Bağlayıcı arşivleri soldan sağa tarar ve yalnızca o ana dek çözülmemiş sembollerini alır.

31.2 Statik Kütüphane Oluşturma (.a)

Statik kütüphane, nesne dosyalarının ar ile paketlenmiş bir arşividir. Bağlama sırasında gereken nesneler *ikiliye kopyalanır*.

```
1 $ gcc -c topla.c carp.c      # topla.o, carp.o uret
2 $ ar rcs libmat.a topla.o carp.o  # arşivle (r=ekle c=olustur s=indeks)
3 $ ar t libmat.a              # icindekileri listele
4
5 $ gcc main.c -L. -lmat -o main  # -L. : bu dizinde ara, -lmat : libmat.a
```

31.3 Paylaşımlı Kütüphane Oluşturma (.so)

Paylaşımlı kütüphane çalışma zamanında bağlanır; kod ikiliye kopyalanmaz, birden çok program aynı `.so`'yu paylaşır. Konumdan bağımsız kod (`-fPIC`) şarttır.

```
1 $ gcc -fPIC -c topla.c carp.c
2 $ gcc -shared -Wl,-soname,libmat.so.1 -o libmat.so.1.0.0 topla.o carp.o
3 $ ln -sf libmat.so.1.0.0 libmat.so.1      # soname -> gercek dosya
```

```

4 $ ln -sf libmat.so.1      libmat.so      # baglama adi (-lmat bunu bulur)
5
6 $ gcc main.c -L. -lmat -o main          # baglama zamani
7 $ ./main                                # HATA: libmat.so.1 bulunamadi
8 $ LD_LIBRARY_PATH=. ./main              # calisma zamani yolunu ver

```

soname, rpath ve ldconfig

soname (libmat.so.1) ABI sürümünü kodlar; yalnızca uyumsuz değişimde artırılır. Yolu ikiliye gömmek için gcc ... -Wl,-rpath,'\$ORIGIN' (ikilinin yanındaki .so'yu bulur). Sisteme kalıcı kurulum için .so'yu /usr/local/lib'e koyup sudo ldconfig çalıştır.

31.4 Sık Kullanılan Bağlama Bayrakları

Bayrak	Görev
-Wl,-Map=out.map	Bellek/sembol haritası dosyası üret (neyin nereye gittiği).
-ffunction-sections -fdata-sections	Her fonksiyon/veriyi ayrı section'a koy (aşağıdakiyle birlikte).
-Wl,-gc-sections	Hiç kullanılmayan section'ları at (ikili küçülür).
-Wl,-wrap=malloc	malloc çağrılarını __wrap_malloc'a yönlendir (test/mock).
-Wl,-Bstatic -lfoo -Wl,-Bdynamic	Yalnız foo'yu statik, gerisini dinamik bağla.
-Wl,-as-needed	Faillen kullanılmayan .so bağımlılıklarını ekleme.
-nostdlib / -nostartfiles	libc / crt nesnelerini ekleme (gömülü, çekirdek, freestanding).
-Wl,-version-script=v.map	Hangi sembollerin dışa açık (global) olacağını denetle.
-Wl,-T,link.ld	Özel linker script kullan (aşağıya bkz.).

-wrap ile fonksiyon sarmalama

Bir sembolü kendi kodunla değiştir; gerçeğine __real_ önekiyle eriş. Ayırma sayacı takmak için birebir.

```

1 #include <stdlib.h>
2 extern void *__real_malloc(size_t); // gercek malloc
3 void *__wrap_malloc(size_t n) {    // tum malloc cagrilari buraya gelir
4     void *p = __real_malloc(n);
5     /* burada log/sayac ekleyebilirsiniz */
6     return p;
7 }
8 // Derleme: gcc main.c wrap.c -Wl,--wrap=malloc -o main

```

31.5 Linker Script Nedir, Ne Zaman Gerekir?

Linker script (genelde .ld uzantılı), bağlayıcıya section'ları hangi bellek adreslerine yerleştireceğini anlatan bir metin dosyasıdır. ld'nin zaten dahili (varsayılan) bir script'i vardır; sıradan Linux uygulamalarında ona dokunmana gerek yoktur. **Kendi script'ini** şu durumlarda yazarsın:

- Gömülü/bare-metal (STM32 vb.): Flash ve RAM adreslerini fiziksel olarak sen belirlemelisin.
- Çekirdek/bootloader: ENTRY noktası ve segment adresleri kesin olmalı.
- Özel section'lar: Belirli veriyi bilinen bir adrese sabitlemek.

```
1 $ ld --verbose # ld'nin VARSAYILAN linker script'ini bas (ogrenmek icin)
```

31.6 Minimal Linker Script Anatomisi

Üç temel yapı taşı vardır: ENTRY (giriş sembolü), MEMORY (fiziksel bellek bölgeleri) ve SECTIONS (çıktı section'larının yerleşimi). . sembolü “konum sayacıdır” (location counter) — yerleştirdikçe ilerler.

```
1  /* link.ld --- gomulu bir hedef icin ornek */
2  ENTRY(_start) /* giris noktası sembolu */
3
4  MEMORY { /* fiziksel bellek haritası */
5      FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 512K
6      RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
7  }
8
9  SECTIONS {
10     .text : { /* kod + salt-okunur veri -> FLASH */
11         *(.text*) /* tum girdi .text section'lari */
12         *(.rodata*)
13     } > FLASH
14
15     .data : { /* ilklenmis veri: RAM'de calisir... */
16         _sdata = .; /* baslangic adresini sembole yaz */
17         *(.data*)
18         _edata = .;
19     } > RAM AT > FLASH /* ...ama ilk degerleri FLASH'ta durur (LMA) */
20
21     .bss : { /* sifirlanacak veri -> RAM */
22         _sbss = .;
23         *(.bss*)
24         _ebss = .;
25     } > RAM
26 }
```

```
1 # Kullanım (gomulu, libc'siz):
2 $ arm-none-eabi-gcc -T link.ld -nostdlib start.o main.o -o firmware.elf
3 # Linux'ta gcc surucusuyle:
4 $ gcc -T link.ld main.o -o program
```

Bilgi

Script içinde tanımlanan `_sdata`, `_ebss` gibi semboller C tarafından extern olarak görülür. Başlangıç kodu (startup) bunları kullanarak `.data`'yı FLASH'tan RAM'e kopyalar ve `.bss`'i sıfırlar — yani main'den önce belleği “kuran” koddur. Ayrıntılı bir gömülü örnek için *STM32/ARM Gömülü* rehberindeki linker script bölümüne bakabilirsin.

VMA vs LMA

VMA (Virtual Memory Address) section'ın çalışırken bulunacağı adres; **LMA** (Load Memory Address) ise başlangıçta *yüklendiği* adrestir. > RAM AT > FLASH kalıbı “çalışması RAM’de ama ilk değeri FLASH’ta” demektir; farkı startup kodu kopyalayarak kapatır.

31.7 Sembol Görünürlüğünü Denetleme (version script)

Bir paylaşımlı kütüphanede hangi sembollerin dışarıya açık olacağını version script ile sınırlarsın — ABI’yi temiz tutar, iç sembolleri gizler.

```
1 /* dinamik semboller: sadece api_* dışarı açık, gerisi gizli */
2 {
3     global:
4         api_*;
5     local:
6         *;
7 };
```

```
1 $ gcc -shared -fPIC -Wl,--version-script=exports.map -o libapi.so *.o
2 $ nm -D libapi.so          # dışarı açık (dynamic) sembolleri doğrula
```

İpucu

Aynı işi kaynak tarafında `__attribute__((visibility("hidden")))` ve `gcc -fvisibility=hidden` ile de yaparsın; büyük kütüphanelerde bağlama süresini ve sembol çakışmalarını azaltır.

32. Ek: Derleme, Hata Ayıklama ve Araçlar

Komut	Görev
<code>gcc -Wall -Wextra -g</code>	Uyarılar + hata ayıklama sembolleri.
<code>gcc -fsanitize=address</code>	Bellek hatalarını çalışma anında yakala.
<code>gcc -pthread</code>	POSIX threads bağla.
<code>gdb ./program</code>	Etkileşimli hata ayıklayıcı.
<code>valgrind ./program</code>	Bellek sızıntısı/hatası dedektörü.
<code>strace ./program</code>	Yapılan sistem çağrılarını izle.
<code>ltrace ./program</code>	Kütüphane çağrılarını izle.
<code>man 2 / man 3</code>	Sistem çağrısı / kütüphane el kitabı.

Tipik bir hata ayıklama oturumu

Segfault’u gdb ile bulmak.

```
1 $ gcc -g -O0 program.c -o program          # -g şart, -O0 optimizasyonu kapat
2 $ gdb ./program
```



```
3 (gdb) run # cokene kadar calistir
4 (gdb) backtrace # cagri yigini: nerede cektu?
5 (gdb) frame 1 # bir ust cerceveye gec
6 (gdb) print variable # bir degiskenin degerini gor
7 (gdb) break dosya.c:42 # 42. satira kesme noktası
8 (gdb) continue # devam et
```

Bilgi

Öğrenmeye devam: man sayfaları en güvenilir kaynaktır. Bu rehberdeki her sistem çağrısının detaylı davranışı, hata kodları ve örnekleri için `man 2 <ad>` komutuna başvurun. Kod örneklerini kendi makinende derleyip `strace` ile hangi sistem çağrılarını yaptıklarını izlemek, sistem programlamayı gerçekten kavramanın en iyi yoludur.